CHAPTER 1

Basic Simulation Modeling

Recommended sections for a first reading: 1.1 through 1.4 (except 1.4.7), 1.7, 1.8

1.1 THE NATURE OF SIMULATION

This is a book about techniques for using computers to imitate, or *simulate*, the operations of various kinds of real-world facilities or processes. The facility or process of interest is usually called a *system*, and in order to study it scientifically we often have to make a set of assumptions about how it works. These assumptions, which usually take the form of mathematical or logical relationships, constitute a *model* that is used to try to gain some understanding of how the corresponding system behaves.

If the relationships that compose the model are simple enough, it may be possible to use mathematical methods (such as algebra, calculus, or probability theory) to obtain *exact* information on questions of interest; this is called an *analytic* solution. However, most real-world systems are too complex to allow realistic models to be evaluated analytically, and these models must be studied by means of simulation. In a *simulation* we use a computer to evaluate a model *numerically*, and data are gathered in order to *estimate* the desired true characteristics of the model.

As an example of the use of simulation, consider a manufacturing company that is contemplating building a large extension on to one of its plants but is not sure if the potential gain in productivity would justify the construction cost. It certainly would not be cost-effective to build the extension and then remove it later if it does not work out. However, a careful simulation study could shed some light on the question by simulating the operation of the plant as it currently exists and as it *would* be *if* the plant were expanded.

Application areas for simulation are numerous and diverse. Below is a list of some particular kinds of problems for which simulation has been found to be a useful and powerful tool:

- Designing and analyzing manufacturing systems
- Evaluating military weapons systems or their logistics requirements
- Determining hardware requirements or protocols for communications networks
- Determining hardware and software requirements for a computer system
- Designing and operating transportation systems such as airports, freeways, ports, and subways
- Evaluating designs for service organizations such as call centers, fast-food restaurants, hospitals, and post offices
- Reengineering of business processes
- Analyzing supply chains
- Determining ordering policies for an inventory system
- Analyzing mining operations

Simulation is one of the most widely used operations-research and management-science techniques, if not *the* most widely used. One indication of this is the Winter Simulation Conference, which attracts 600 to 800 people every year. In addition, there are several other simulation conferences that often have more than 100 participants per year.

There are also several surveys related to the use of operations-research techniques. For example, Lane, Mansour, and Harpell (1993) reported from a longitudinal study, spanning 1973 through 1988, that simulation was consistently ranked as one of the three most important "operations-research techniques." The other two were "math programming" (a catch-all term that includes many individual techniques such as linear programming, nonlinear programming, etc.) and "statistics" (which is not an operations-research technique per se). Gupta (1997) analyzed 1294 papers from the journal *Interfaces* (one of the leading journals dealing with applications of operations research) from 1970 through 1992, and found that simulation was second only to "math programming" among 13 techniques considered.

There have been, however, several impediments to even wider acceptance and usefulness of simulation. First, models used to study large-scale systems tend to be very complex, and writing computer programs to execute them can be an arduous task indeed. This task has been made much easier in recent years by the development of excellent software products that automatically provide many of the features needed to "program" a simulation model. A second problem with simulation of complex systems is that a large amount of computer time is sometimes required. However, this difficulty has become much less severe as computers become faster and cheaper. Finally, there appears to be an unfortunate impression that simulation is just an exercise in computer programming, albeit a complicated one. Consequently, many simulation "studies" have been composed of heuristic model building, programming, and a single run of the program to obtain "the answer." We fear that this attitude, which neglects the important issue of how a properly coded model should be used to make inferences about the system of interest, has doubtless led to erroneous conclusions being drawn from

many simulation studies. These questions of simulation *methodology*, which are largely independent of the software and hardware used, form an integral part of the latter chapters of this book.

Perspectives on the historical evolution of simulation modeling may be found in Nance and Sargent (2002).

In the remainder of this chapter (as well as in Chap. 2) we discuss systems and models in considerably greater detail and then show how to write computer programs in a general-purpose language to simulate systems of varying degrees of complexity. All of the computer code shown in this chapter can be downloaded from www.mhhe.com/law.

1.2 SYSTEMS, MODELS, AND SIMULATION

A *system* is defined to be a collection of entities, e.g., people or machines, that act and interact together toward the accomplishment of some logical end. [This definition was proposed by Schmidt and Taylor (1970).] In practice, what is meant by "the system" depends on the objectives of a particular study. The collection of entities that comprise a system for one study might be only a subset of the overall system for another. For example, if one wants to study a bank to determine the number of tellers needed to provide adequate service for customers who want just to cash a check or make a savings deposit, the system can be defined to be that portion of the bank consisting of the tellers and the customers waiting in line or being served. If, on the other hand, the loan officer and the safe-deposit boxes are to be included, the definition of the system must be expanded in an obvious way. [See also Fishman (1978, p. 3).] We define the *state* of a system to be that collection of variables necessary to describe a system at a particular time, relative to the objectives of a study. In a study of a bank, examples of possible state variables are the number of busy tellers, the number of customers in the bank, and the time of arrival of each customer in the bank.

We categorize systems to be of two types, discrete and continuous. A *discrete* system is one for which the state variables change instantaneously at separated points in time. A bank is an example of a discrete system, since state variables—e.g., the number of customers in the bank—change only when a customer arrives or when a customer finishes being served and departs. A *continuous* system is one for which the state variables change continuously with respect to time. An airplane moving through the air is an example of a continuous system, since state variables such as position and velocity can change continuously with respect to time. Few systems in practice are wholly discrete or wholly continuous; but since one type of change predominates for most systems, it will usually be possible to classify a system as being either discrete or continuous.

At some point in the lives of most systems, there is a need to study them to try to gain some insight into the relationships among various components, or to predict performance under some new conditions being considered. Figure 1.1 maps out different ways in which a system might be studied.

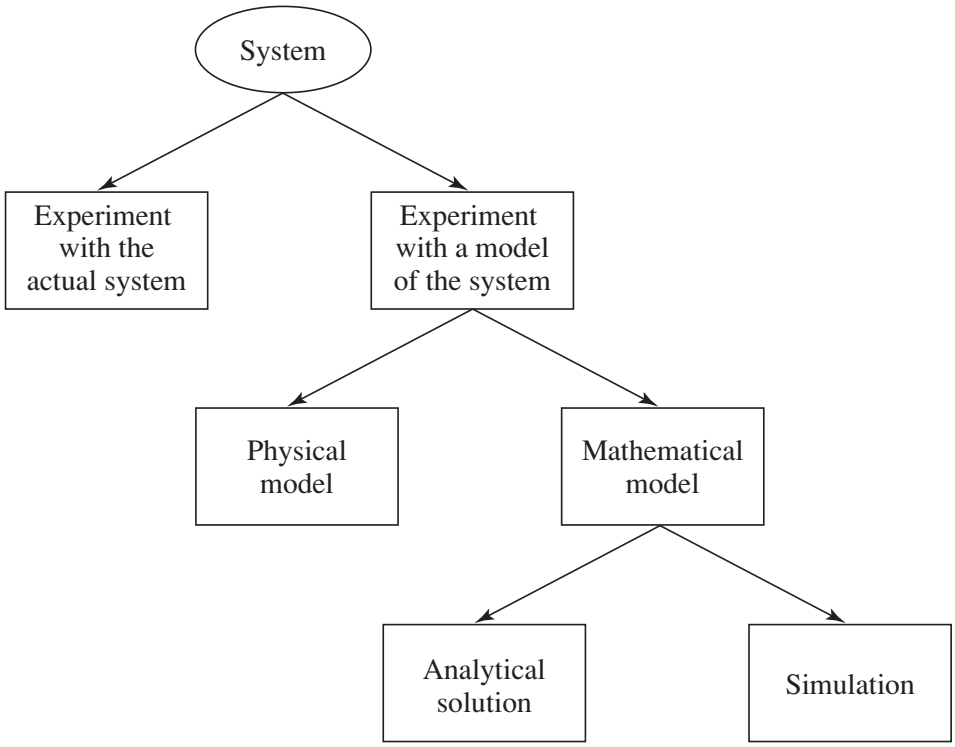


FIGURE 1.1
Ways to study a system.

- *Experiment with the Actual System vs. Experiment with a Model of the System.* If it is possible (and cost-effective) to alter the system physically and then let it operate under the new conditions, it is probably desirable to do so, for in this case there is no question about whether what we study is valid. However, it is rarely feasible to do this, because such an experiment would often be too costly or too disruptive to the system. For example, a bank may be contemplating reducing the number of tellers to decrease costs, but actually trying this could lead to long customer delays and alienation. More graphically, the "system" might not even exist, but we nevertheless want to study it in its various proposed alternative configurations to see how it should be built in the first place; examples of this situation might be a proposed communications network, or a strategic nuclear weapons system. For these reasons, it is usually necessary to build a *model* as a representation of the system and study it as a surrogate for the actual system. When using a model, there is always the question of whether it accurately reflects the system for the purposes of the decisions to be made; this question of model *validity* is taken up in detail in Chap. 5.
- *Physical Model vs. Mathematical Model.* To most people, the word "model" evokes images of clay cars in wind tunnels, cockpits disconnected from their airplanes to be used in pilot training, or miniature supertankers scurrying about in a swimming pool. These are examples of *physical* models (also called *iconic* models), and are not typical of the kinds of models that are usually of interest in operations research and systems analysis. Occasionally, however, it has been found useful to build physical models to study engineering or management

systems; examples include tabletop scale models of material-handling systems, and in at least one case a full-scale physical model of a fast-food restaurant inside a warehouse, complete with full-scale, real (and presumably hungry) humans [see Swart and Donno (1981)]. But the vast majority of models built for such purposes are *mathematical*, representing a system in terms of logical and quantitative relationships that are then manipulated and changed to see how the model reacts, and thus how the system *would* react—*if* the mathematical model is a valid one. Perhaps the simplest example of a mathematical model is the familiar relation $d = rt$, where r is the rate of travel, t is the time spent traveling, and d is the distance traveled. This might provide a valid model in one instance (e.g., a space probe to another planet after it has attained its flight velocity) but a very poor model for other purposes (e.g., rush-hour commuting on congested urban freeways).

- *Analytical Solution vs. Simulation.* Once we have built a mathematical model, it must then be examined to see how it can be used to answer the questions of interest about the system it is supposed to represent. If the model is simple enough, it may be possible to work with its relationships and quantities to get an exact, *analytical* solution. In the $d = rt$ example, if we know the distance to be traveled and the velocity, then we can work with the model to get $t = d/r$ as the time that will be required. This is a very simple, closed-form solution obtainable with just paper and pencil, but some analytical solutions can become extraordinarily complex, requiring vast computing resources; inverting a large nonsparse matrix is a well-known example of a situation in which there is an analytical formula known in principle, but obtaining it numerically in a given instance is far from trivial. If an analytical solution to a mathematical model is available and is computationally efficient, it is usually desirable to study the model in this way rather than via a simulation. However, many systems are highly complex, so that valid mathematical models of them are themselves complex, precluding any possibility of an analytical solution. In this case, the model must be studied by means of *simulation*, i.e., numerically exercising the model for the inputs in question to see how they affect the output measures of performance.

While there may be a small element of truth to pejorative old saws such as "method of last resort" sometimes used to describe simulation, the fact is that we are very quickly led to simulation in most situations, due to the sheer complexity of the systems of interest and of the models necessary to represent them in a valid way.

Given, then, that we have a mathematical model to be studied by means of simulation (henceforth referred to as a *simulation model*), we must then look for particular tools to do this. It is useful for this purpose to classify simulation models along three different dimensions:

- *Static vs. Dynamic Simulation Models.* A *static* simulation model is a representation of a system at a particular time, or one that may be used to represent a system in which time simply plays no role; examples of static simulations are certain Monte Carlo models, discussed in Sec. 13.5. On the other hand, a *dynamic* simulation model represents a system as it evolves over time, such as a conveyor system in a factory.

- *Deterministic vs. Stochastic Simulation Models*. If a simulation model does not contain any probabilistic (i.e., random) components, it is called *deterministic*; a complicated (and analytically intractable) system of differential equations describing a chemical reaction might be such a model. In deterministic models, the output is "determined" once the set of input quantities and relationships in the model have been specified, even though it might take a lot of computer time to evaluate what it is. Many systems, however, must be modeled as having at least some random input components, and these give rise to *stochastic* simulation models. (For an example of the danger of ignoring randomness in modeling a system, see Sec. 4.7.) Most queueing and inventory systems are modeled stochastically. Stochastic simulation models produce output that is itself random, and must therefore be treated as only an estimate of the true characteristics of the model; this is one of the main disadvantages of simulation (see Sec. 1.8) and is dealt with in Chaps. 9 through 12 of this book.
- *Continuous vs. Discrete Simulation Models.* Loosely speaking, we define *discrete* and *continuous* simulation models analogously to the way discrete and continuous systems were defined above. More precise definitions of discrete (event) simulation and continuous simulation are given in Secs. 1.3 and 13.3, respectively. It should be mentioned that a discrete model is not always used to model a discrete system, and vice versa. The decision whether to use a discrete or a continuous model for a particular system depends on the specific objectives of the study. For example, a model of traffic flow on a freeway would be discrete if the characteristics and movement of individual cars are important. Alternatively, if the cars can be treated "in the aggregate," the flow of traffic can be described by differential equations in a continuous model. More discussion on this issue can be found in Sec. 5.2, and in particular in Example 5.2.

The simulation models we consider in the remainder of this book, except for those in Chap. 13, will be discrete, dynamic, and stochastic and will henceforth be called *discrete-event simulation models*. (Since deterministic models are a special case of stochastic models, the restriction to stochastic models involves no loss of generality.)

1.3 DISCRETE-EVENT SIMULATION

Discrete-event simulation concerns the modeling of a system as it evolves over time by a representation in which the state variables change instantaneously at separate points in time. (In more mathematical terms, we might say that the system can change at only a *countable* number of points in time.) These points in time are the ones at which an event occurs, where an *event* is defined as an instantaneous occurrence that may change the state of the system. Although discrete-event simulation could conceptually be done by hand calculations, the amount of data that must be stored and manipulated for most real-world systems dictates that discrete-event simulations be done on a digital computer. (In Sec. 1.4.2 we carry out a small hand simulation, merely to illustrate the logic involved.)

EXAMPLE 1.1. Consider a service facility with a single server—e.g., a one-operator barbershop or an information desk at an airport—for which we would like to estimate the (expected) average delay in queue (line) of arriving customers, where the delay in queue of a customer is the length of the time interval from the instant of his arrival at the facility to the instant he begins being served. For the objective of estimating the average delay of a customer, the state variables for a discrete-event simulation model of the facility would be the status of the server, i.e., either idle or busy, the number of customers waiting in queue to be served (if any), and the time of arrival of each person waiting in queue. The status of the server is needed to determine, upon a customer's arrival, whether the customer can be served immediately or must join the end of the queue. When the server completes serving a customer, the number of customers in the queue is used to determine whether the server will become idle or begin serving the first customer in the queue. The time of arrival of a customer is needed to compute his delay in queue, which is the time he begins being served (which will be known) minus his time of arrival. There are two types of events for this system: the arrival of a customer and the completion of service for a customer, which results in the customer's departure. An arrival is an event since it causes the (state variable) server status to change from idle to busy or the (state variable) number of customers in the queue to increase by 1. Correspondingly, a departure is an event because it causes the server status to change from busy to idle or the number of customers in the queue to decrease by 1. We show in detail how to build a discrete-event simulation model of this single-server queueing system in Sec. 1.4.

In the above example both types of events actually changed the state of the system, but in some discrete-event simulation models events are used for purposes that do not actually effect such a change. For example, an event might be used to schedule the end of a simulation run at a particular time (see Sec. 1.4.6) or to schedule a decision about a system's operation at a particular time (see Sec. 1.5) and might not actually result in a change in the state of the system. This is why we originally said that an event *may* change the state of a system.

1.3.1 Time-Advance Mechanisms

Because of the dynamic nature of discrete-event simulation models, we must keep track of the current value of simulated time as the simulation proceeds, and we also need a mechanism to advance simulated time from one value to another. We call the variable in a simulation model that gives the current value of simulated time the *simulation clock*. The unit of time for the simulation clock is never stated explicitly when a model is written in a general-purpose language such as C, and it is assumed to be in the same units as the input parameters. Also, there is generally no relationship between simulated time and the time needed to run a simulation on the computer.

Historically, two principal approaches have been suggested for advancing the simulation clock: *next-event time advance* and *fixed-increment time advance.* Since the first approach is used by all major simulation software and by most people programming their model in a general-purpose language, and since the second is a special case of the first, we shall use the next-event time-advance approach for all discrete-event simulation models discussed in this book. A brief discussion of fixed-increment time advance is given in App. 1A (at the end of this chapter).

With the next-event time-advance approach, the simulation clock is initialized to zero and the times of occurrence of future events are determined. The simulation clock is then advanced to the time of occurrence of the *most imminent* (first) of these future events, at which point the state of the system is updated to account for the fact that an event has occurred, and our knowledge of the times of occurrence of future events is also updated. Then the simulation clock is advanced to the time of the (new) most imminent event, the state of the system is updated, and future event times are determined, etc. This process of advancing the simulation clock from one event time to another is continued until eventually some prespecified stopping condition is satisfied. Since all state changes occur only at event times for a discrete-event simulation model, periods of inactivity are skipped over by jumping the clock from event time to event time. (Fixed-increment time advance does not skip over these inactive periods, which can eat up a lot of computer time; see App. 1A.) It should be noted that the successive jumps of the simulation clock are generally variable (or unequal) in size.

EXAMPLE 1.2. We now illustrate in detail the next-event time-advance approach for the single-server queueing system of Example 1.1. We need the following notation:

t_i = time of arrival of the ith customer ($t_0 = 0$)
$A_i = t_i - t_{i-1}$ = interarrival time between $(i-1)$st and ith arrivals of customers
S_i = time that server actually spends serving ith customer (exclusive of customer's delay in queue)
D_i = delay in queue of ith customer
$c_i = t_i + D_i + S_i$ = time that ith customer completes service and departs
e_i = time of occurrence of ith event of any type (ith value the simulation clock takes on, excluding the value $e_0 = 0$)

Each of these defined quantities will generally be a random variable. Assume that the probability distributions of the interarrival times $A_1, A_2, \ldots$ and the service times $S_1, S_2, \ldots$ are known and have cumulative distribution functions (see Sec. 4.2) denoted by F_A and F_S, respectively. (In general, F_A and F_S would be determined by collecting data from the system of interest and then specifying distributions consistent with these data using the techniques of Chap. 6.) At time $e_0 = 0$ the status of the server is idle, and the time t_1 of the first arrival is determined by generating A_1 from F_A (techniques for generating random observations from a specified distribution are discussed in Chap. 8) and adding it to 0. The simulation clock is then advanced from e_0 to the time of the next (first) event, $e_1 = t_1$. (See Fig. 1.2, where the curved arrows represent advancing the simulation clock.) Since the customer arriving at time t_1 finds the server idle, she immediately enters service and has a delay in queue of $D_1 = 0$ and the status of the server is changed from idle to busy. The time, c_1, when the arriving customer will complete service is computed by generating S_1 from F_S and adding it to t_1. Finally, the time of the second arrival, t_2, is computed as $t_2 = t_1 + A_2$, where A_2 is generated from F_A. If $t_2 < c_1$, as depicted in Fig. 1.2, the simulation clock is advanced from e_1 to the time of the next event, $e_2 = t_2$. (If c_1 were less than t_2, the clock would be advanced from e_1 to c_1.) Since the customer arriving at time t_2 finds the server already busy, the number of customers in the queue is increased from 0 to 1 and the time of arrival of this customer is recorded; however, his service time S_2 is not generated at this time. Also, the time of the third arrival, t_3, is computed as $t_3 = t_2 + A_3$. If $c_1 < t_3$, as depicted in the figure, the simulation clock is advanced from e_2 to the time of the next event, $e_3 = c_1$, where the customer

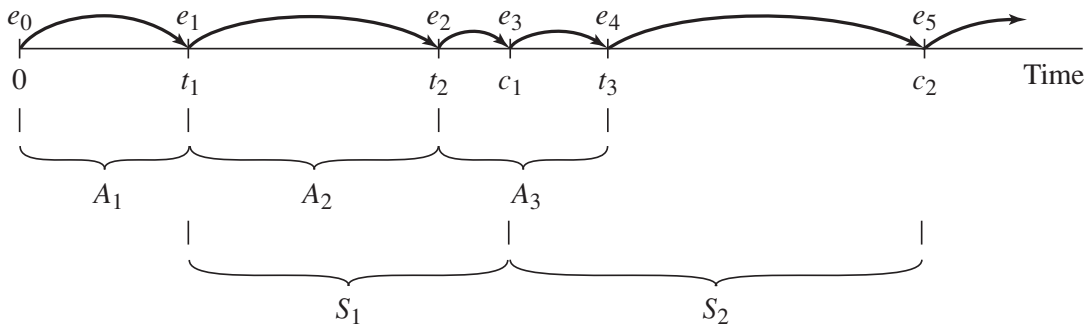


FIGURE 1.2
The next-event time-advance approach illustrated for the single-server queueing system.

completing service departs, the customer in the queue (i.e., the one who arrived at time t_2) begins service and his delay in queue and service-completion time are computed as $D_2 = c_1 - t_2$ and $c_2 = c_1 + S_2$ (S_2 is now generated from F_S), and the number of customers in the queue is decreased from 1 to 0. If $t_3 < c_2$, the simulation clock is advanced from e_3 to the time of the next event, $e_4 = t_3$, etc. The simulation might eventually be terminated when, say, the number of customers whose delays have been observed reaches some specified value.

1.3.2 Components and Organization of a Discrete-Event Simulation Model

Although simulation has been applied to a great diversity of real-world systems, discrete-event simulation models all share a number of common components and there is a logical organization for these components that promotes the programming, debugging, and future changing of a simulation model's computer program. In particular, the following components will be found in most discrete-event simulation models using the next-event time-advance approach programmed in a general-purpose language:

System state: The collection of state variables necessary to describe the system at a particular time

Simulation clock: A variable giving the current value of simulated time

Event list: A list containing the next time when each type of event will occur

Statistical counters: Variables used for storing statistical information about system performance

Initialization routine: A subprogram to initialize the simulation model at time 0

Timing routine: A subprogram that determines the next event from the event list and then advances the simulation clock to the time when that event is to occur

Event routine: A subprogram that updates the system state when a particular type of event occurs (there is one event routine for each event type)

Library routines: A set of subprograms used to generate random observations from probability distributions that were determined as part of the simulation model

Report generator: A subprogram that computes estimates (from the statistical counters) of the desired measures of performance and produces a report when the simulation ends

Main program: A subprogram that invokes the timing routine to determine the next event and then transfers control to the corresponding event routine to update the system state appropriately. The main program may also check for termination and invoke the report generator when the simulation is over.

The logical relationships (flow of control) among these components are shown in Fig. 1.3. The simulation begins at time 0 with the main program invoking the initialization routine, where the simulation clock is set to zero, the system state and the statistical counters are initialized, and the event list is initialized. After control has been returned to the main program, it invokes the timing routine to determine which type of event is most imminent. If an event of type i is the next to occur, the simulation clock is advanced to the time that event type i will occur

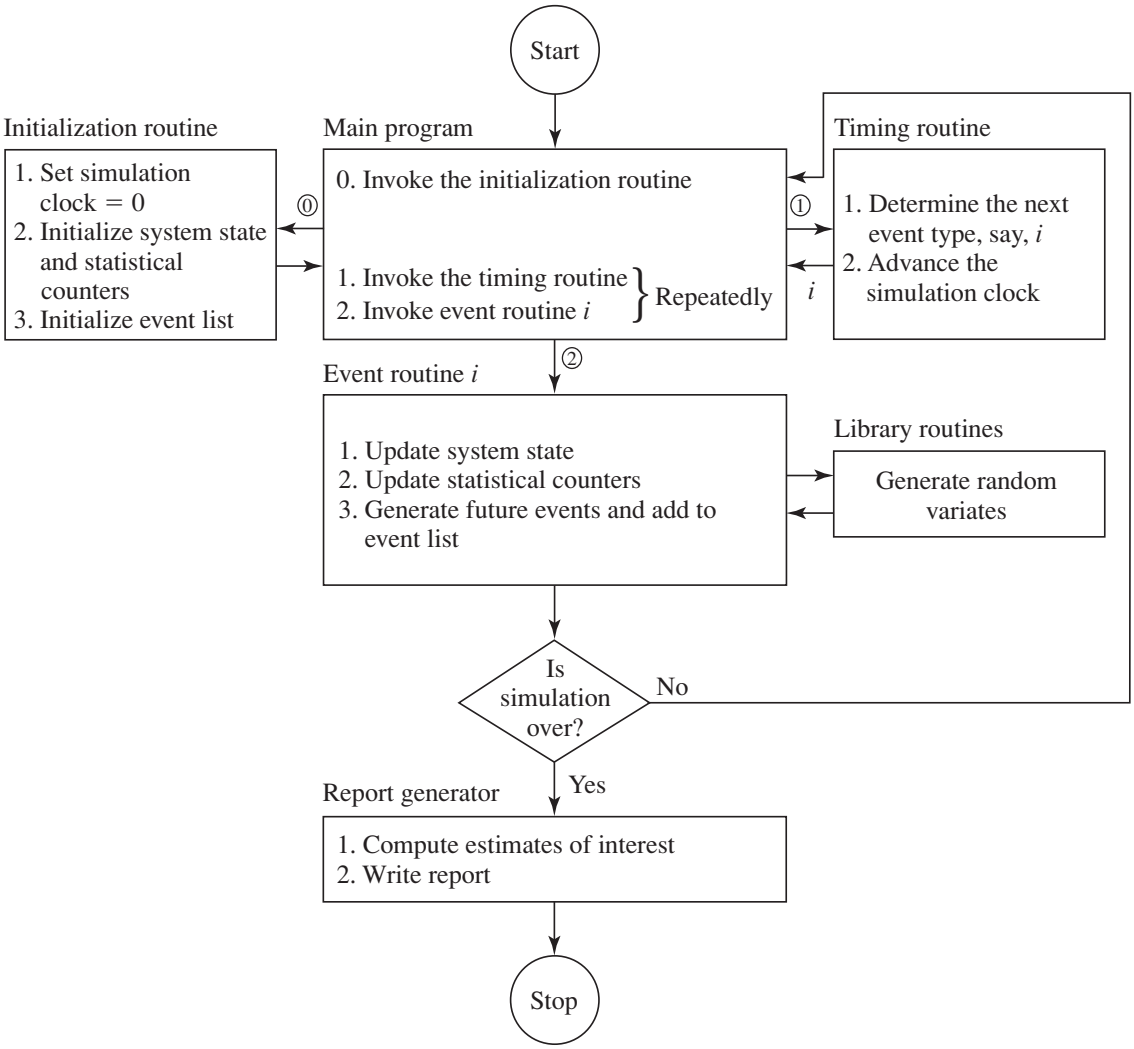


FIGURE 1.3
Flow of control for the next-event time-advance approach.

and control is returned to the main program. Then the main program invokes event routine *i*, where typically three types of activities occur: (1) The system state is updated to account for the fact that an event of type *i* has occurred; (2) information about system performance is gathered by updating the statistical counters; and (3) the times of occurrence of future events are generated, and this information is added to the event list. Often it is necessary to generate random observations from probability distributions in order to determine these future event times; we will refer to such a generated observation as a *random variate*. After all processing has been completed, either in event routine *i* or in the main program, a check is typically made to determine (relative to some stopping condition) if the simulation should now be terminated. If it is time to terminate the simulation, the report generator is invoked from the main program to compute estimates (from the statistical counters) of the desired measures of performance and to produce a report. If it is not time for termination, control is passed back to the main program and the main program–timing routine–main program–event routine–termination check cycle is repeated until the stopping condition is eventually satisfied.

Before concluding this section, a few additional words about the system state may be in order. As mentioned in Sec. 1.2, a system is a well-defined collection of *entities*. Entities are characterized by data values called *attributes*, and these attributes are part of the system state for a discrete-event simulation model. Furthermore, entities with some common property are often grouped together in *lists* (or *files* or *sets*). For each entity there is a *record* in the list consisting of the entity's attributes, and the order in which the records are placed in the list depends on some specified rule. (See Chap. 2 for a discussion of efficient approaches for storing lists of records.) For the single-server queueing facility of Examples 1.1 and 1.2, the entities are the server and the customers in the facility. The server has the attribute "server status" (busy or idle), and the customers waiting in queue have the attribute "time of arrival." (The number of customers in the queue might also be considered an attribute of the server.) Furthermore, as we shall see in Sec. 1.4, these customers in queue will be grouped together in a list.

The organization and action of a discrete-event simulation program using the next-event time-advance mechanism as depicted above are fairly typical when programming such simulations in a general-purpose programming language such as C; it is called the *event-scheduling approach* to simulation modeling, since the times of future events are explicitly coded into the model and are scheduled to occur in the simulated future. It should be mentioned here that there is an alternative approach to simulation modeling, called the *process approach*, that instead views the simulation in terms of the individual entities involved, and the code written describes the "experience" of a "typical" entity as it "flows" through the system; programming simulations modeled from the process point of view usually requires the use of special-purpose simulation software, as discussed in Chap. 3. Even when taking the process approach, however, the simulation is actually executed behind the scenes in the event-scheduling logic as described above.

1.4 SIMULATION OF A SINGLE-SERVER QUEUEING SYSTEM

This section shows in detail how to simulate a single-server queueing system such as a one-operator barbershop. Although this system seems very simple compared with those usually of real interest, how it is simulated is actually quite representative of the operation of simulations of great complexity.

In Sec. 1.4.1 we describe the system of interest and state our objectives more precisely. We explain intuitively how to simulate this system in Sec. 1.4.2 by showing a "snapshot" of the simulated system just after each event occurs. Section 1.4.3 describes the language-independent organization and logic of the C code given in Sec. 1.4.4. The simulation's results are discussed in Sec. 1.4.5, and Sec. 1.4.6 alters the stopping rule to another common way to end simulations. Finally, Sec. 1.4.7 briefly describes a technique for identifying and simplifying the event and variable structure of a simulation.

1.4.1 Problem Statement

Consider a single-server queueing system (see Fig. 1.4) for which the interarrival times $A_1, A_2, \ldots$ are *independent and identically distributed* (IID) random variables.

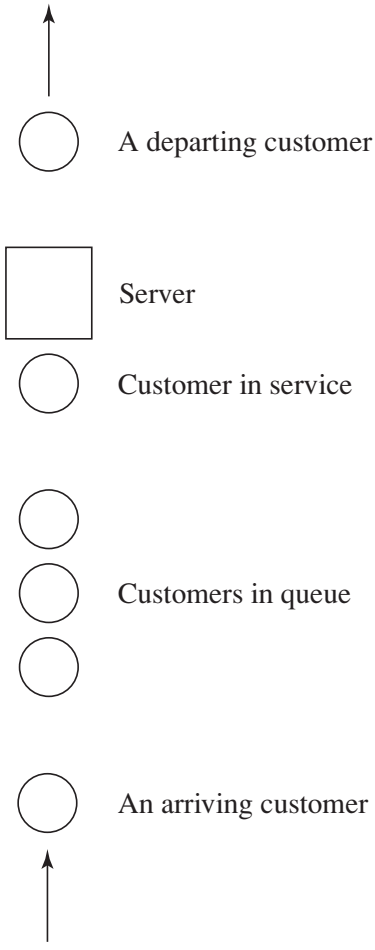


FIGURE 1.4
A single-server queueing system.

("Identically distributed" means that the interarrival times have the same probability distribution.) A customer who arrives and finds the server idle enters service immediately, and the service times $S_1, S_2, \ldots$ of the successive customers are IID random variables that are independent of the interarrival times. A customer who arrives and finds the server busy joins the end of a single queue. Upon completing service for a customer, the server chooses a customer from the queue (if any) in a first-in, first-out (FIFO) manner. (For a discussion of other queue disciplines and queueing systems in general, see App. 1B.)

The simulation will begin in the "empty-and-idle" state; i.e., no customers are present and the server is idle. At time 0, we will begin waiting for the arrival of the first customer, which will occur after the first interarrival time, A_1, rather than at time 0 (which would be a possibly valid, but different, modeling assumption). We wish to simulate this system until a fixed number (n) of customers have completed their delays in queue; i.e., the simulation will stop when the nth customer enters service. Note that the *time* the simulation ends is thus a random variable, depending on the observed values for the interarrival and service-time random variables.

To measure the performance of this system, we will look at estimates of three quantities. First, we will estimate the expected average delay in queue of the n customers completing their delays during the simulation; we denote this quantity by $d(n)$. The word "expected" in the definition of $d(n)$ means this: On a given run of the simulation (or, for that matter, on a given run of the actual system the simulation model represents), the actual average delay observed of the n customers depends on the interarrival and service-time random variable observations that happen to have been obtained. On another run of the simulation (or on a different day for the real system) there would probably be arrivals at different times, and the service times required would also be different; this would give rise to a different value for the average of the n delays. Thus, the average delay on a given run of the simulation is properly regarded as a random variable itself. What we want to estimate, $d(n)$, is the *expected value* of this random variable. One interpretation of this is that $d(n)$ is the average of a large (actually, infinite) number of n-customer average delays. From a single run of the simulation resulting in customer delays $D_1, D_2, \ldots, D_n$, an obvious estimator of $d(n)$ is

$$\hat{d}(n) = \frac{\sum_{i=1}^{n} D_i}{n}$$

which is just the average of the n D_i's that were observed in the simulation [so that $\hat{d}(n)$ could also be denoted by $\overline{D}(n)$]. [Throughout this book, a hat (ˆ) above a symbol denotes an estimator.] It is important to note that by "delay" we do not exclude the possibility that a customer could have a delay of zero in the case of an arrival finding the system empty and idle (with this model, we know for sure that $D_1 = 0$); delays with a value of 0 *are* counted in the average, since if many delays were zero this would represent a system providing very good service, and our output measure should reflect this. One reason for taking the average of the D_i's, as opposed to just looking at them individually, is that they will not have the same distribution (e.g., $D_1 = 0$, but D_2 could be positive), and the average gives us a single composite measure of all

the customers' delays; in this sense, this is not the usual "average" taken in basic statistics, as the individual terms are not independent random observations from the same distribution. Note also that by itself, $\hat{d}(n)$ is an estimator based on a sample of size *1*, since we are making only one complete simulation run. From elementary statistics, we know that a sample of size 1 is not worth much; we return to this issue in Chaps. 9 through 12.

While an estimate of $d(n)$ gives information about system performance from the customers' point of view, the management of such a system may want different information; indeed, since most real simulations are quite complex and may be time-consuming to run, we usually collect many output measures of performance, describing different aspects of system behavior. One such measure for our simple model here is the expected average number of customers in the queue (but not being served), denoted by $q(n)$, where the n is necessary in the notation to indicate that this average is taken over the time period needed to observe the n delays defining our stopping rule. This is a different kind of "average" than the average delay in queue, because it is taken over (continuous) time, rather than over customers (being discrete). Thus, we need to define what is meant by this *time*-average number of customers in queue. To do this, let $Q(t)$ denote the number of customers in queue at time t, for any real number $t \geq 0$, and let $T(n)$ be the time required to observe our n delays in queue. Then for any time t between 0 and $T(n)$, $Q(t)$ is a nonnegative integer. Further, if we let p_i be the expected *proportion* (which will be between 0 and 1) of the time that $Q(t)$ is equal to i, then a reasonable definition of $q(n)$ would be

$$q(n) = \sum_{i=0}^{\infty} ip_i$$

Thus, $q(n)$ is a weighted average of the possible values i for the queue length $Q(t)$, with the weights being the expected proportion of time the queue spends at each of its possible lengths. To estimate $q(n)$ from a simulation, we simply replace the p_i's with estimates of them, and get

$$\hat{q}(n) = \sum_{i=0}^{\infty} i\hat{p}_i \tag{1.1}$$

where $\hat{p}_i$ is the *observed* (rather than expected) proportion of the time *during the simulation* that there were i customers in the queue. Computationally, however, it is easier to rewrite $\hat{q}(n)$ using some geometric considerations. If we let T_i be the *total* time during the simulation that the queue is of length i, then $T(n) = T_0 + T_1 + T_2 + \cdots$ and $\hat{p}_i = T_i/T(n)$, so that we can rewrite Eq. (1.1) above as

$$\hat{q}(n) = \frac{\sum_{i=0}^{\infty} iT_i}{T(n)} \tag{1.2}$$

Figure 1.5 illustrates a possible time path, or *realization*, of $Q(t)$ for this system in the case of $n = 6$; ignore the shading for now. Arrivals occur at times 0.4, 1.6, 2.1, 3.8, 4.0, 5.6, 5.8, and 7.2. Departures (service completions) occur at times 2.4, 3.1, 3.3, 4.9, and 8.6, and the simulation ends at time $T(6) = 8.6$. Remember in looking

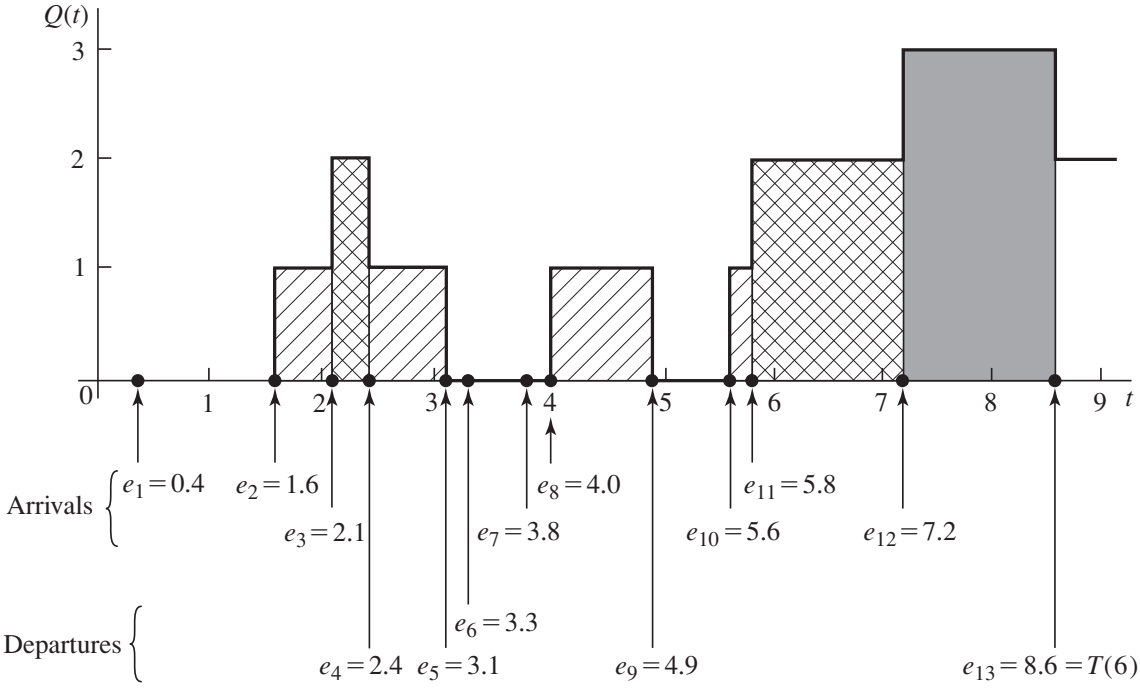


FIGURE 1.5
$Q(t)$, arrival times, and departure times for a realization of a single-server queueing system.

at Fig. 1.5 that $Q(t)$ does not count the customer in service (if any), so between times 0.4 and 1.6 there is one customer in the system being served, even though the queue is empty [$Q(t) = 0$]; the same is true between times 3.1 and 3.3, between times 3.8 and 4.0, and between times 4.9 and 5.6. Between times 3.3 and 3.8, however, the system is empty of customers and the server is idle, as is obviously the case between times 0 and 0.4. To compute $\hat{q}(n)$, we must first compute the T_i's, which can be read off Fig. 1.5 as the (sometimes separated) intervals over which $Q(t)$ is equal to 0, 1, 2, and so on:

$$T_0 = (1.6 - 0.0) + (4.0 - 3.1) + (5.6 - 4.9) = 3.2$$
$$T_1 = (2.1 - 1.6) + (3.1 - 2.4) + (4.9 - 4.0) + (5.8 - 5.6) = 2.3$$
$$T_2 = (2.4 - 2.1) + (7.2 - 5.8) = 1.7$$
$$T_3 = (8.6 - 7.2) = 1.4$$

($T_i = 0$ for $i \geq 4$, since the queue never grew to those lengths in this realization.) The numerator in Eq. (1.2) is thus

$$\sum_{i=0}^{\infty} iT_i = (0 \times 3.2) + (1 \times 2.3) + (2 \times 1.7) + (3 \times 1.4) = 9.9 \qquad (1.3)$$

and so our estimate of the time-average number in queue from this particular simulation run is $\hat{q}(6) = 9.9/8.6 = 1.15$. Now, note that each of the nonzero terms on the right-hand side of Eq. (1.3) corresponds to one of the shaded areas in Fig. 1.5: 1×2.3 is the diagonally shaded area (in four pieces), 2×1.7 is the cross-hatched

area (in two pieces), and 3×1.4 is the screened area (in a single piece). In other words, the summation in the numerator of Eq. (1.2) is just the *area under the $Q(t)$ curve between the beginning and the end of the simulation*. Remembering that "area under a curve" is an integral, we can thus write

$$\sum_{i=0}^{\infty} iT_i = \int_0^{T(n)} Q(t)\, dt$$

and the estimator of $q(n)$ can then be expressed as

$$\hat{q}(n) = \frac{\displaystyle\int_0^{T(n)} Q(t)\, dt}{T(n)} \tag{1.4}$$

While Eqs. (1.4) and (1.2) are equivalent expressions for $\hat{q}(n)$, Eq. (1.4) is preferable since the integral in this equation can be accumulated as simple areas of rectangles as the simulation progresses through time. It is less convenient to carry out the computations to get the summation in Eq. (1.2) explicitly. Moreover, the appearance of Eq. (1.4) suggests a continuous average of $Q(t)$, since in a rough sense, an integral can be regarded as a continuous summation.

The third and final output measure of performance for this system is a measure of how busy the server is. The expected *utilization* of the server is the expected proportion of time during the simulation [from time 0 to time $T(n)$] that the server is busy (i.e., not idle), and is thus a number between 0 and 1; denote it by $u(n)$. From a single simulation, then, our estimate of $u(n)$ is $\hat{u}(n)$ = the *observed* proportion of time during the simulation that the server is busy. Now $\hat{u}(n)$ could be computed directly from the simulation by noting the times at which the server changes status (idle to busy or vice versa) and then doing the appropriate subtractions and division. However, it is easier to look at this quantity as a continuous-time average, similar to the average queue length, by defining the "busy function"

$$B(t) = \begin{cases} 1 & \text{if the server is busy at time } t \\ 0 & \text{if the server is idle at time } t \end{cases}$$

and so $\hat{u}(n)$ could be expressed as the proportion of time that $B(t)$ is equal to 1. Figure 1.6 plots $B(t)$ for the same simulation realization as used in Fig. 1.5 for $Q(t)$. In this case, we get

$$\hat{u}(n) = \frac{(3.3 - 0.4) + (8.6 - 3.8)}{8.6} = \frac{7.7}{8.6} = 0.90 \tag{1.5}$$

indicating that the server was busy about 90 percent of the time during this simulation. Again, however, the numerator in Eq. (1.5) can be viewed as the area under the $B(t)$ function over the course of the simulation, since the height of $B(t)$ is always either 0 or 1. Thus,

$$\hat{u}(n) = \frac{\displaystyle\int_0^{T(n)} B(t)\, dt}{T(n)} \tag{1.6}$$

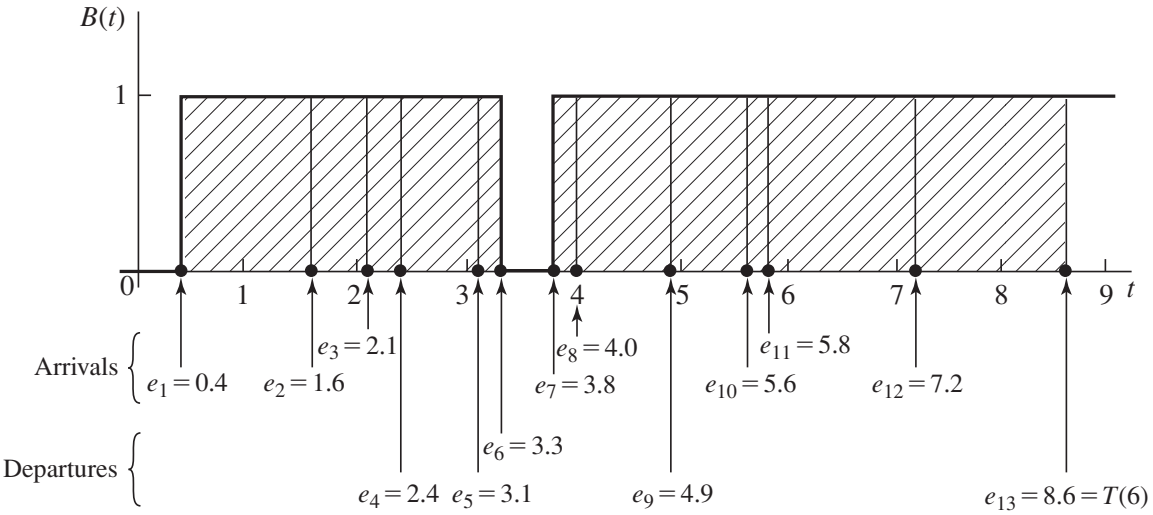


FIGURE 1.6
$B(t)$, arrival times, and departure times for a realization of a single-server queueing system (same realization as in Fig. 1.5).

and we see again that $\hat{u}(n)$ is the continuous average of the $B(t)$ function, corresponding to our notion of utilization. As was the case for $\hat{q}(n)$, the reason for writing $\hat{u}(n)$ in the integral form of Eq. (1.6) is that computationally, as the simulation progresses, the integral of $B(t)$ can easily be accumulated by adding up areas of rectangles. For many simulations involving "servers" of some sort, utilization statistics are quite informative in identifying bottlenecks (utilizations near 100 percent, coupled with heavy congestion measures for the queue leading in) or excess capacity (low utilizations); this is particularly true if the "servers" are expensive items such as robots in a manufacturing system or large mainframe computers in a data-processing operation.

To recap, the three measures of performance are the average delay in queue $\hat{d}(n)$, the time-average number of customers in queue $\hat{q}(n)$, and the proportion of time the server is busy $\hat{u}(n)$. The average delay in queue is an example of a *discrete-time statistic*, since it is defined relative to the collection of random variables $\{D_i\}$ that have a discrete "time" index, $i = 1, 2, \ldots$. The time-average number in queue and the proportion of time the server is busy are examples of *continuous-time statistics*, since they are defined on the collection of random variables $\{Q(t)\}$ and $\{B(t)\}$, respectively, each of which is indexed on the continuous time parameter $t \in [0, \infty)$. (The symbol $\in$ means "contained in." Thus, in this case, t can be any nonnegative real number.) Both discrete-time and continuous-time statistics are common in simulation, and they furthermore can be other than averages. For example, we might be interested in the *maximum* of all the delays in queue observed (a discrete-time statistic), or the *proportion* of time during the simulation that the queue contained at least five customers (a continuous-time statistic).

The events for this system are the arrival of a customer and the departure of a customer (after a service completion); the state variables necessary to estimate $d(n)$, $q(n)$, and $u(n)$ are the status of the server (0 for idle and 1 for busy), the number of customers in the queue, the time of arrival of each customer currently in the queue

(represented as a list), and the time of the last (i.e., most recent) event. The time of the last event, defined to be e_{i-1} if $e_{i-1} \le t < e_i$ (where t is the current time in the simulation), is needed to compute the width of the rectangles for the area accumulations in the estimates of $q(n)$ and $u(n)$.

1.4.2 Intuitive Explanation

We begin our explanation of how to simulate a single-server queueing system by showing how its simulation model would be represented inside the computer at time $e_0 = 0$ and the times $e_1, e_2, \ldots, e_{13}$ at which the 13 successive events occur that are needed to observe the desired number, $n = 6$, of delays in queue. For expository convenience, we assume that the interarrival and service times of customers are

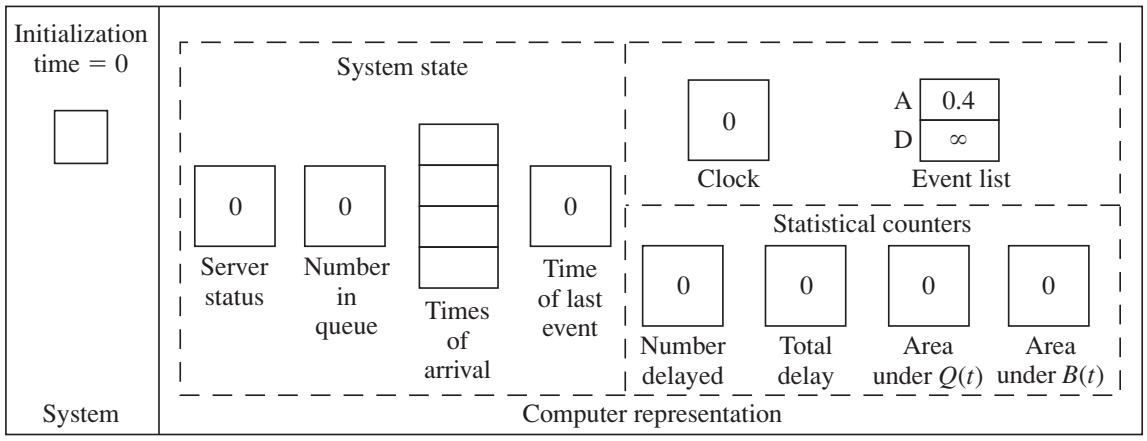
$$A_1 = 0.4, A_2 = 1.2, A_3 = 0.5, A_4 = 1.7, A_5 = 0.2,$$
$$A_6 = 1.6, A_7 = 0.2, A_8 = 1.4, A_9 = 1.9, \ldots$$

$$S_1 = 2.0, S_2 = 0.7, S_3 = 0.2, S_4 = 1.1, S_5 = 3.7, S_6 = 0.6, \ldots$$

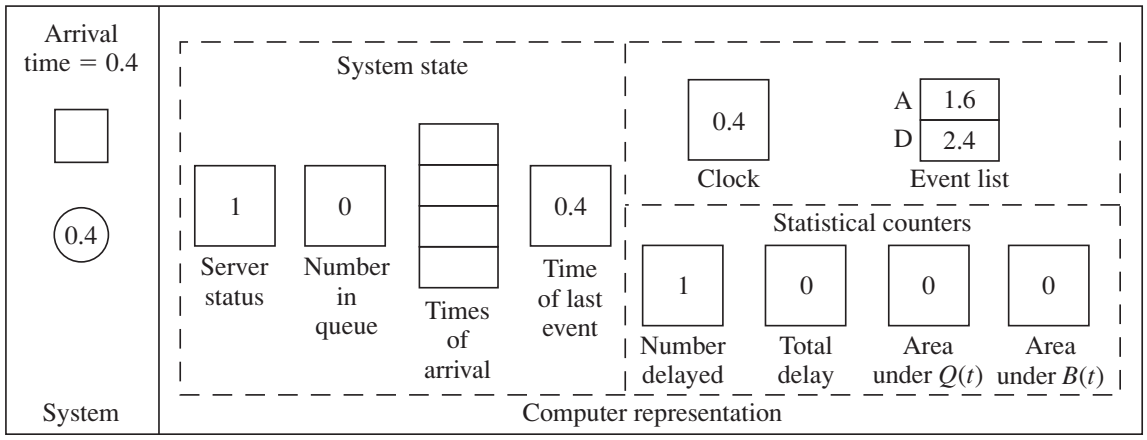
Thus, between time 0 and the time of the first arrival there is 0.4 time unit, between the arrivals of the first and second customers there are 1.2 time units, etc., and the service time required for the first customer is 2.0 time units, etc. Note that it is not necessary to declare what the time units are (minutes, hours, etc.), but only to be sure that all time quantities are expressed in the *same* units. In an actual simulation (see Sec. 1.4.4), the A_i's and the S_i's would be generated from their corresponding probability distributions, as needed, during the course of the simulation. The numerical values for the A_i's and the S_i's given above have been artificially chosen so as to generate the same simulation realization as depicted in Figs. 1.5 and 1.6 illustrating the $Q(t)$ and $B(t)$ processes.

Figure 1.7 gives a snapshot of the system itself and of a computer representation of the system at each of the times $e_0 = 0, e_1 = 0.4, \ldots, e_{13} = 8.6$. In the "system" pictures, the square represents the server, and circles represent customers; the numbers inside the customer circles are the times of their arrivals. In the "computer representation" pictures, the values of the variables shown are *after* all processing has been completed at that event. Our discussion will focus on how the computer representation changes at the event times.

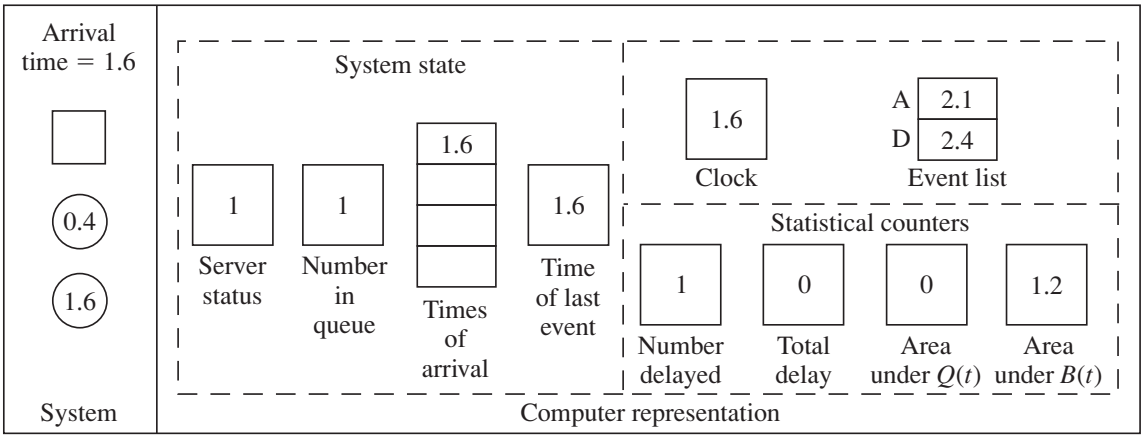
$t = 0$: *Initialization.* The simulation begins with the main program invoking the initialization routine. Our modeling assumption was that initially the system is empty of customers and the server is idle, as depicted in the "system" picture of Fig. 1.7*a*. The model state variables are initialized to represent this: Server status is 0 [we use 0 to represent an idle server and 1 to represent a busy server, similar to the definition of the $B(t)$ function], and the number of customers in the queue is 0. There is a one-dimensional array to store the times of arrival of customers *currently in the queue*; this array is initially empty, and as the simulation progresses, its length will grow and shrink. The time of the last (most recent) event is initialized to 0, so that at the time of the first event (when it is used), it will have its correct value. The simulation



(*a*)

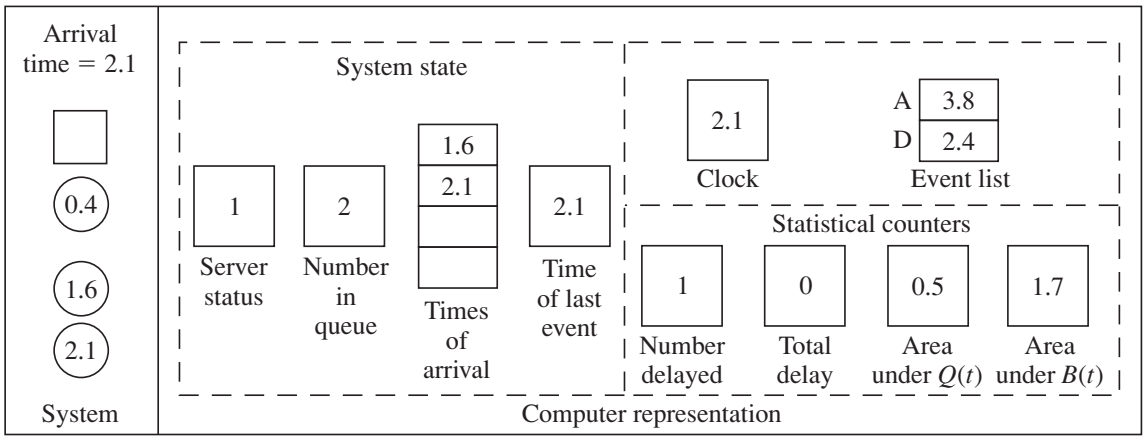


(*b*)

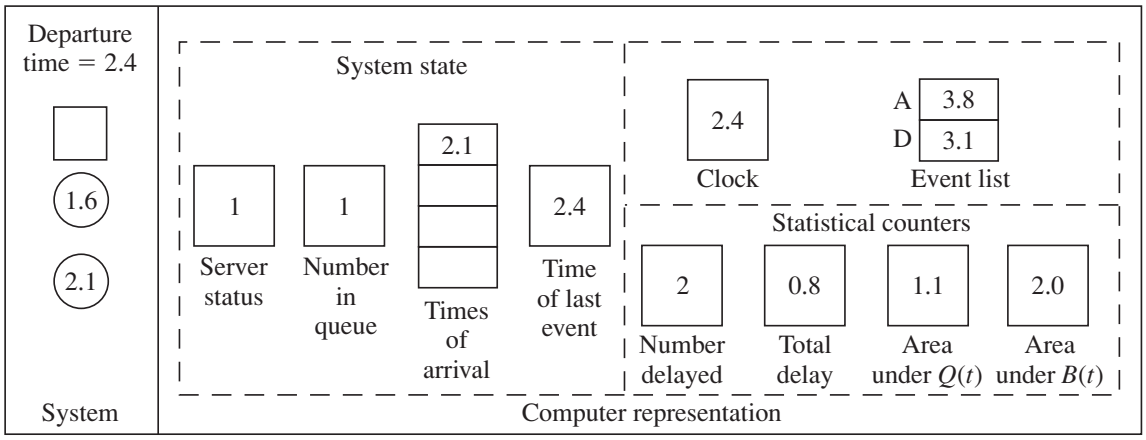


(*c*)

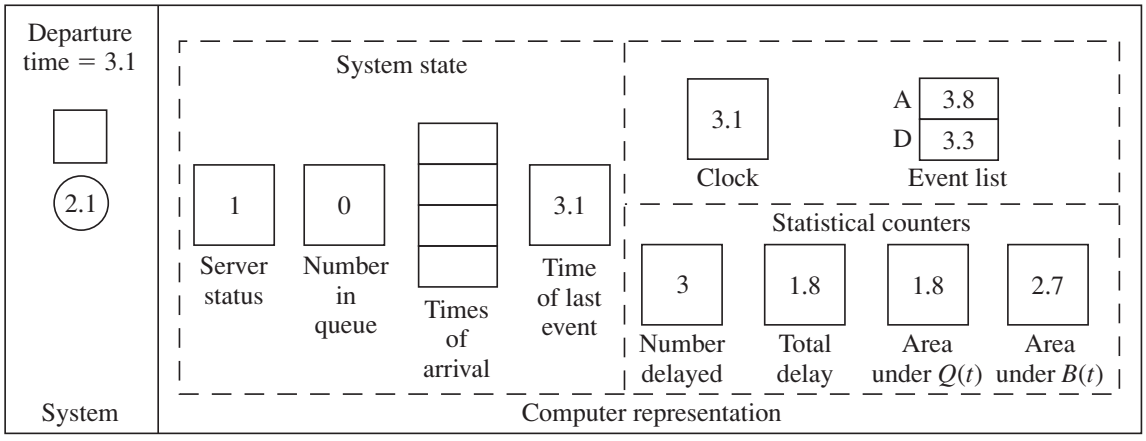
FIGURE 1.7
Snapshots of the system and of its computer representation at time 0 and at each of the 13 succeeding event times.



(*d*)

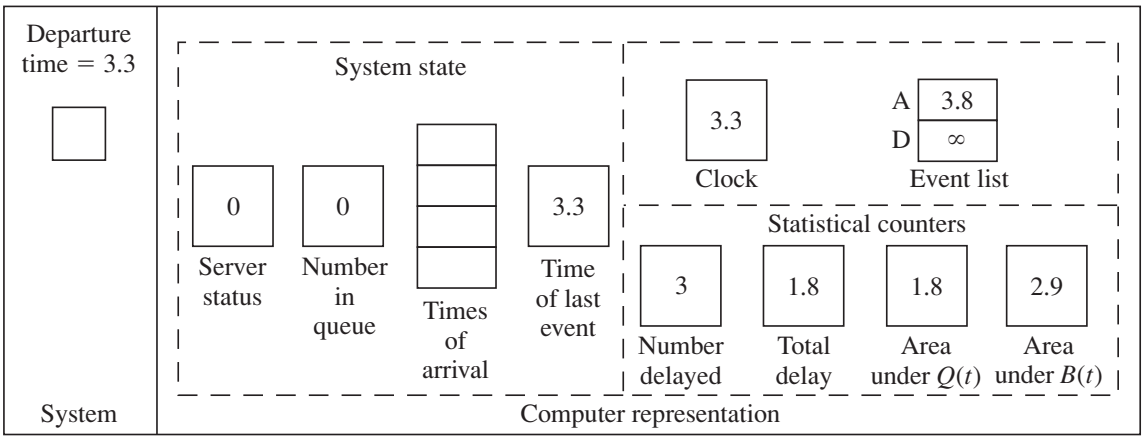


(*e*)

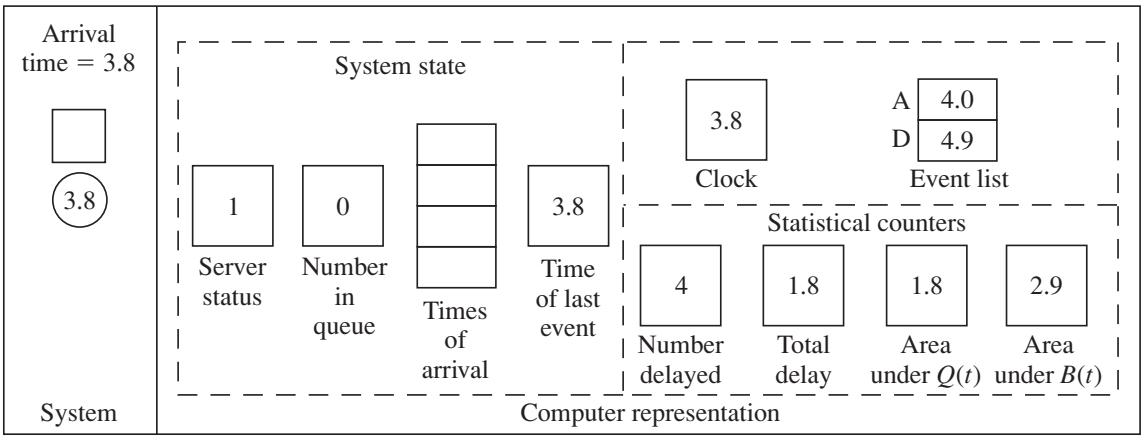


(*f*)

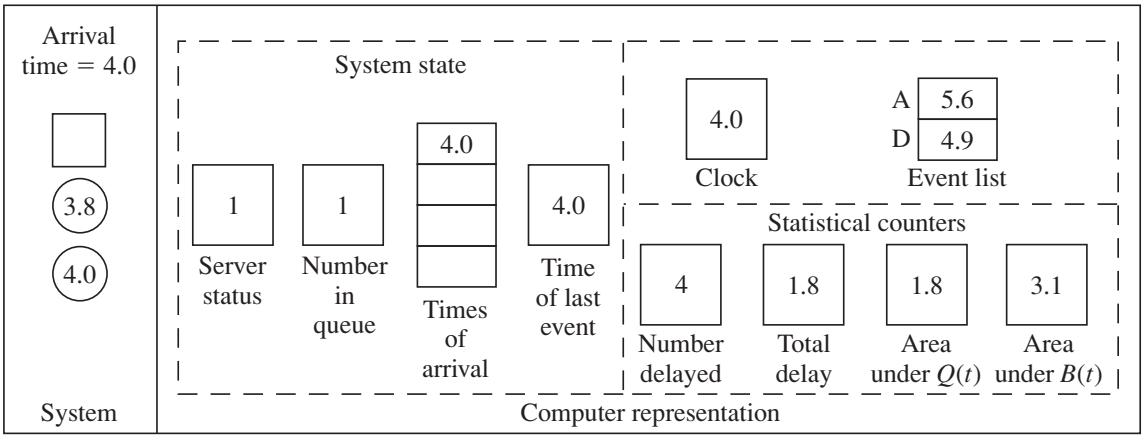
FIGURE 1.7
(*continued*)



(*g*)

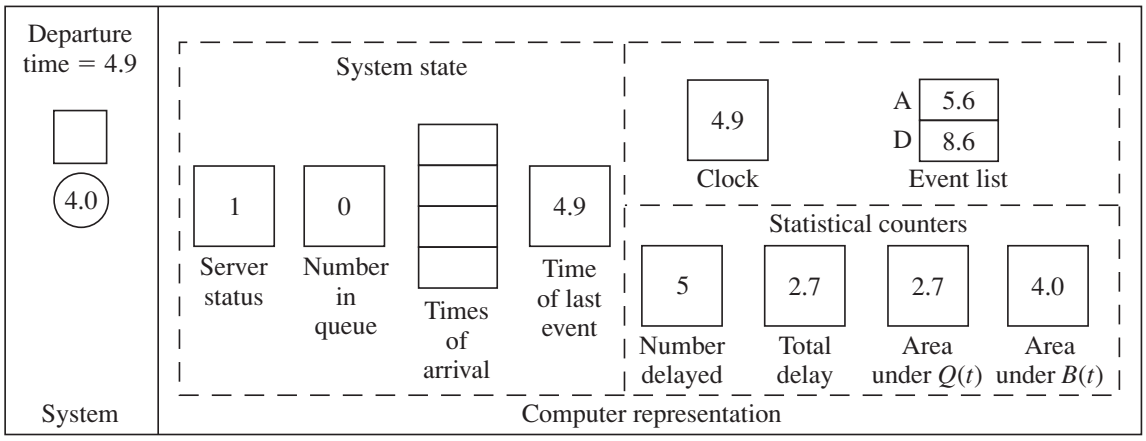


(*h*)

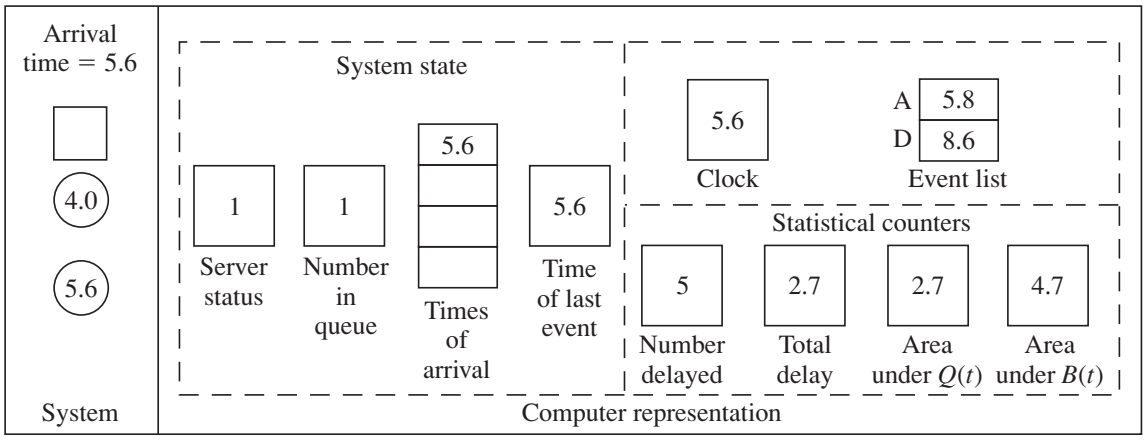


(*i*)

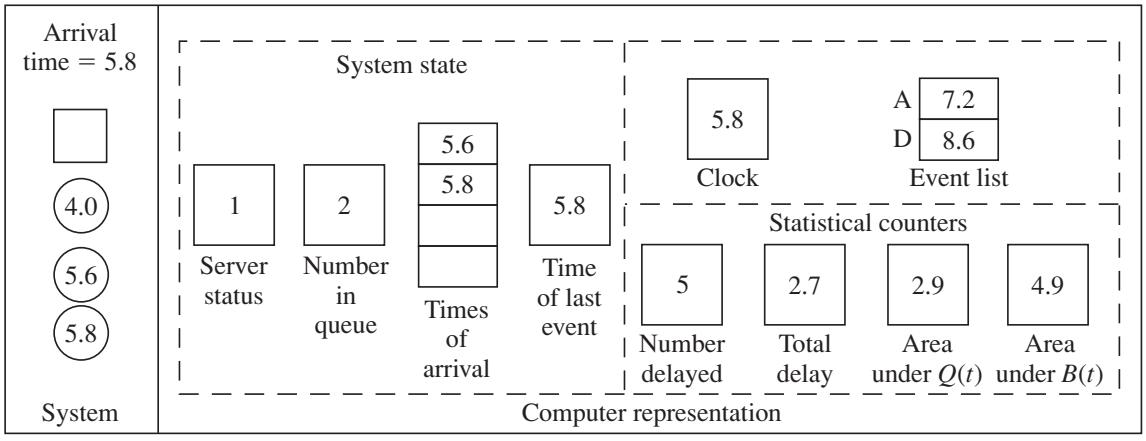
FIGURE 1.7
(*continued*)



(*j*)

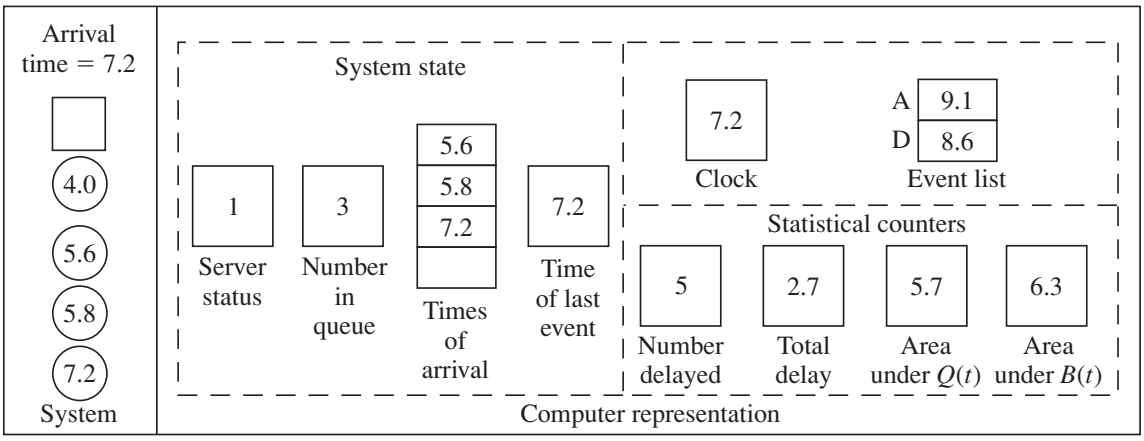


(*k*)

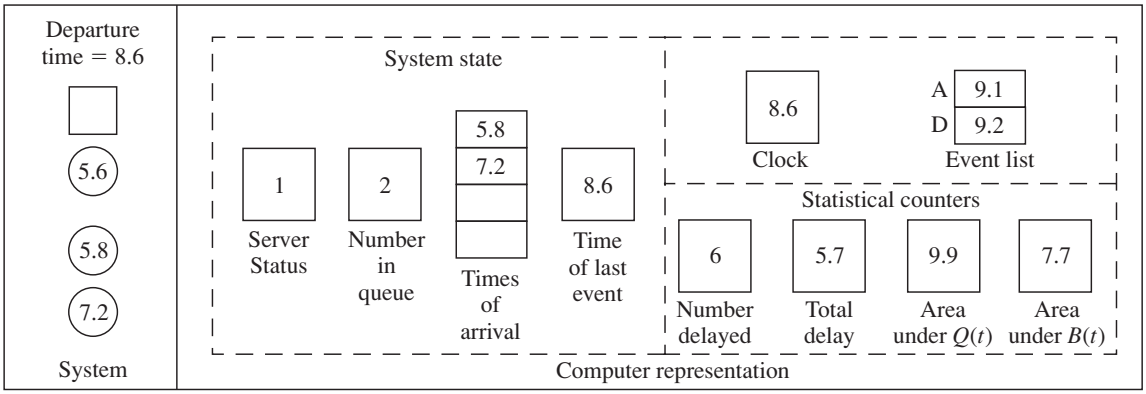


(*l*)

FIGURE 1.7
(*continued*)



(*m*)



(*n*)

FIGURE 1.7
(*continued*)

clock is set to 0, and the *event list*, giving the times of the next occurrence of each of the event types, is initialized as follows. The time of the first arrival is $0 + A_1 = 0.4$, and is denoted by "A" next to the event list. Since there is no customer in service, it does not even make sense to talk about the time of the next departure ("D" by the event list), and we know that the first event will be the initial customer arrival at time 0.4. However, the simulation progresses in general by looking at the event list and picking the smallest value from it to determine what the next event will be, so by scheduling the next departure to occur at time ∞ (or a very large number in a computer program), we effectively eliminate the departure event from consideration and force the next event to be an arrival. Finally, the four statistical counters are initialized to 0. When all initialization is done, control is returned to the main program, which then calls the timing routine to determine the next event. Since $0.4 < \infty$, the next event will be an arrival at time 0.4, and the

timing routine advances the clock to this time, then passes control back to the main program with the information that the next event is to be an arrival.

$t = 0.4$: *Arrival of customer 1.* At time 0.4, the main program passes control to the arrival routine to process the arrival of the first customer. Figure 1.7*b* shows the system and its computer representation *after* all changes have been made to process this arrival. Since this customer arrived to find the server idle (status equal to 0), he begins service immediately and has a delay in queue of $D_1 = 0$ (which *does* count as a delay). The server status is set to 1 to represent that the server is now busy, but the queue itself is still empty. The clock has been advanced to the current time, 0.4, and the event list is updated to reflect this customer's arrival: The next arrival will be $A_2 = 1.2$ time units from now, at time $0.4 + 1.2 = 1.6$, and the next departure (the service completion of the customer now arriving) will be $S_1 = 2.0$ time units from now, at time $0.4 + 2.0 = 2.4$. The number delayed is incremented to 1 (when this reaches $n = 6$, the simulation will end), and $D_1 = 0$ is added into the total delay (still at zero). The area under $Q(t)$ is updated by adding in the product of the *previous* value (i.e., the level it had between the last event and now) of $Q(t)$ (0 in this case) times the width of the interval of time from the last event to now, $t -$ (time of last event) $= 0.4 - 0$ in this case. Note that the time of the last event used here is its *old* value (0), before it is updated to its new value (0.4) in this event routine. Similarly, the area under $B(t)$ is updated by adding in the product of its previous value (0) times the width of the interval of time since the last event. [Look back at Figs. 1.5 and 1.6 to trace the accumulation of the areas under $Q(t)$ and $B(t)$.] Finally, the time of the last event is brought up to the current time, 0.4, and control is passed back to the main program. It invokes the timing routine, which scans the event list for the smallest value, and determines that the next event will be another arrival at time 1.6; it updates the clock to this value and passes control back to the main program with the information that the next event is an arrival.

$t = 1.6$: *Arrival of customer 2.* At this time we again enter the arrival routine, and Fig. 1.7*c* shows the system and its computer representation after all changes have been made to process this event. Since this customer arrives to find the server busy (status equal to 1 upon her arrival), she must queue up in the first location in the queue, her time of arrival is stored in the first location in the array, and the number-in-queue variable rises to 1. The time of the next arrival in the event list is updated to $A_3 = 0.5$ time unit from now, $1.6 + 0.5 = 2.1$; the time of the next departure is not changed, since its value of 2.4 is the departure time of customer 1, who is still in service at this time. Since we are not observing the end of anyone's delay in queue, the number-delayed and total-delay variables are unchanged. The area under $Q(t)$ is increased by 0 [the previous value of $Q(t)$] times the time since the last event, $1.6 - 0.4 = 1.2$. The area under $B(t)$ is increased by 1 [the previous value of $B(t)$] times this same interval of time, 1.2. After updating the

time of the last event to now, control is passed back to the main program and then to the timing routine, which determines that the next event will be an arrival at time 2.1.

$t = 2.1$: *Arrival of customer 3.* Once again the arrival routine is invoked, as depicted in Fig. 1.7*d*. The server stays busy, and the queue grows by one customer, whose time of arrival is stored in the queue array's second location. The next arrival is updated to $t + A_4 = 2.1 + 1.7 = 3.8$, and the next departure is still the same, as we are still waiting for the service completion of customer 1. The delay counters are unchanged, since this is not the end of anyone's delay in queue, and the two area accumulators are updated by adding in 1 [the previous values of both $Q(t)$ and $B(t)$] times the time since the last event, $2.1 - 1.6 = 0.5$. After bringing the time of the last event up to the present, we go back to the main program and invoke the timing routine, which looks at the event list to determine that the next event will be a departure at time 2.4, and updates the clock to that time.

$t = 2.4$: *Departure of customer 1.* Now the main program invokes the departure routine, and Fig. 1.7*e* shows the system and its representation after this occurs. The server will maintain its busy status, since customer 2 moves out of the first place in queue and into service. The queue shrinks by 1, and the time-of-arrival array is moved up one place, to represent that customer 3 is now first in line. Customer 2, now entering service, will require $S_2 = 0.7$ time unit, so the time of the next departure (that of customer 2) in the event list is updated to S_2 time units from now, or to time $2.4 + 0.7 = 3.1$; the time of the next arrival (that of customer 4) is unchanged, since this was scheduled earlier at the time of customer 3's arrival, and we are still waiting at this time for customer 4 to arrive. The delay statistics are updated, since at this time customer 2 is entering service and is completing her delay in queue. Here we make use of the time-of-arrival array, and compute the second delay as the current time minus the second customer's time of arrival, or $D_2 = 2.4 - 1.6 = 0.8$. (Note that the value of 1.6 was stored in the first location in the time-of-arrival array *before* it was changed, so this delay computation would have to be done before advancing the times of arrival in the array.) The area statistics are updated by adding in $2 \times (2.4 - 2.1)$ for $Q(t)$ [note that the previous value of $Q(t)$ was used], and $1 \times (2.4 - 2.1)$ for $B(t)$. The time of the last event is updated, we return to the main program, and the timing routine determines that the next event is a departure at time 3.1.

$t = 3.1$: *Departure of customer 2.* The changes at this departure are similar to those at the departure of customer 1 at time 2.4 just discussed. Note that we observe another delay in queue, and that after this event is processed the queue is again empty, but the server is still busy.

$t = 3.3$: *Departure of customer 3.* Again, the changes are similar to those in the above two departure events, with one important exception: Since the queue is now empty, the server becomes idle and we must set the next departure time in the event list to ∞, since the system now looks the

same as it did at time 0 and we want to force the next event to be the arrival of customer 4.

$t = 3.8$: *Arrival of customer 4.* Since this customer arrives to find the server idle, he has a delay of 0 (i.e., $D_4 = 0$) and goes right into service. Thus, the changes here are very similar to those at the arrival of the first customer at time $t = 0.4$.

The remaining six event times are depicted in Fig. 1.7*i* through 1.7*n*, and readers should work through these to be sure they understand why the variables and arrays are as they appear; it may be helpful to follow along in the plots of $Q(t)$ and $B(t)$ in Figs. 1.5 and 1.6. With the departure of customer 5 at time $t = 8.6$, customer 6 leaves the queue and enters service, at which time the number delayed reaches 6 (the specified value of n) and the simulation ends. At this point, the main program invokes the report generator to compute the final output measures [$\hat{d}(6) = 5.7/6 = 0.95$, $\hat{q}(6) = 9.9/8.6 = 1.15$, and $\hat{u}(6) = 7.7/8.6 = 0.90$] and write them out.

A few specific comments about the above example illustrating the logic of a simulation should be made:

- Perhaps the key element in the dynamics of a simulation is the interaction between the simulation clock and the event list. The event list is maintained, and the clock jumps to the next event, as determined by scanning the event list at the end of each event's processing for the smallest (i.e., next) event time. This is how the simulation progresses through time.
- While processing an event, no "simulated" time passes. However, even though time is standing still for the model, care must be taken to process updates of the state variables and statistical counters in the appropriate order. For example, it would be incorrect to update the number in queue before updating the area-under-$Q(t)$ counter, since the height of the rectangle to be used is the *previous* value of $Q(t)$ [before the effect of the current event on $Q(t)$ has been implemented]. Similarly, it would be incorrect to update the time of the last event before updating the area accumulators. Yet another type of error would result if the queue list were changed at a departure before the delay of the first customer in queue were computed, since his time of arrival to the system would be lost.
- It is sometimes easy to overlook contingencies that seem out of the ordinary but that nevertheless must be accommodated. For example, it would be easy to forget that a departing customer could leave behind an empty queue, necessitating that the server be idled and the departure event again be eliminated from consideration. Also, termination conditions are often more involved than they might seem at first sight; in the above example, the simulation stopped in what seems to be the "usual" way, after a departure of one customer, allowing another to enter service and contribute the last delay needed, but the simulation *could* actually have ended instead with an arrival event—how?
- In some simulations it can happen that two (or more) entries in the event list are tied for smallest, and a decision rule must be incorporated to break such *time ties* (this happens with the inventory simulation considered later in Sec. 1.5). The tie-breaking rule can affect the results of the simulation, so must be chosen in accordance with

how the system is to be modeled. In many simulations, however, we can ignore the possibility of ties, since the use of continuous random variables may make their occurrence an event with probability 0. In the above model, for example, if the interarrival-time or service-time distribution is continuous, then a time tie in the event list is a probability-zero event (though it could still happen during the computer simulation due to finite accuracy in representation of real numbers).

The above exercise is intended to illustrate the changes and data structures involved in carrying out a discrete-event simulation from the event-scheduling point of view, and contains most of the important ideas needed for more complex simulations of this type. The interarrival and service times used could have been drawn from a random-number table of some sort, constructed to reflect the desired probability distributions; this would result in what might be called a *hand simulation*, which in principle could be carried out to any length. The tedium of doing this should now be clear, so we will next turn to the use of computers (which are not easily bored) to carry out the arithmetic and bookkeeping involved in longer or more complex simulations.

1.4.3 Program Organization and Logic

In this section we set up the necessary ingredients for the C program to simulate the single-server queueing system, which is given in Sec. 1.4.4.

There are several reasons for choosing a general-purpose language such as C, rather than more powerful high-level simulation software, for introducing computer simulation at this point:

- By learning to simulate in a general-purpose language, in which one must pay attention to every detail, there will be a greater understanding of how simulations actually operate, and thus less chance of conceptual errors if a switch is later made to high-level simulation software.
- Despite the fact that there is now very good and powerful simulation software available (see Chap. 3), it is sometimes necessary to write at least parts of complex simulations in a general-purpose language if the specific, detailed logic of complex systems is to be represented faithfully.
- General-purpose languages are widely available, and entire simulations are sometimes still written in this way.

It is not our purpose in this book to teach any particular simulation software in detail, although we survey several packages in Chap. 3. With the understanding promoted by our more general approach and by going through our simulations in this and the next chapter, the reader should find it easier to learn a specialized simulation-software product.

The single-server queueing model that we will simulate in the following section differs in two respects from the model used in the previous section:

- The simulation will end when $n = 1000$ delays in queue have been completed, rather than $n = 6$, in order to collect more data (and maybe to impress the reader

with the patience of computers, since we have just slugged it out by hand in the $n = 6$ case in the preceding section). It is important to note that this change in the stopping rule changes the model itself, in that the output measures are defined relative to the stopping rule; hence the presence of the "n" in the notation for the quantities $d(n)$, $q(n)$, and $u(n)$ being estimated.

- The interarrival and service times will now be modeled as independent random variables from exponential distributions with mean 1 minute for the interarrival times and mean 0.5 minute for the service times. The exponential distribution with mean β (any positive real number) is continuous, with probability density function

$$f(x) = \frac{1}{\beta} e^{-x/\beta} \qquad \text{for } x \geq 0$$

(See Chaps. 4 and 6 for more information on density functions in general, and on the exponential distribution in particular.) We make this change here since it is much more common to generate input quantities (which drive the simulation) such as interarrival and service times from specified distributions than to assume that they are "known" as we did in the preceding section. The choice of the exponential distribution with the above particular values of β is essentially arbitrary, and is made primarily because it is easy to generate exponential random variates on a computer. (Actually, the assumption of exponential interarrival times is often quite realistic; assuming exponential service times, however, is less plausible.) Chapter 6 addresses in detail the important issue of how one chooses distribution forms and parameters for modeling simulation input random variables.

The single-server queue with exponential interarrival and service times is commonly called the *M/M/1 queue*, as discussed in App. 1B.

To simulate this model, we need a way to generate random variates from an exponential distribution. First, a *random-number generator* (discussed in detail in Chap. 7) is invoked to generate a variate U that is distributed (continuously) uniformly between 0 and 1; this distribution will henceforth be referred to as U(0, 1) and has probability density function

$$f(x) = \begin{cases} 1 & \text{if } 0 \leq x \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

It is easy to show that the probability that a U(0, 1) random variable falls in any subinterval $[x, x + \Delta x]$ contained in the interval [0, 1] is (uniformly) Δx (see Sec. 6.2.2). The U(0, 1) distribution is fundamental to simulation modeling because, as we shall see in Chap. 8, a random variate from any distribution can be generated by first generating one or more U(0, 1) random variates and then performing some kind of transformation. After obtaining U, we shall take the natural logarithm of it, multiply the result by β, and finally change the sign to return what we will show to be an exponential random variate with mean β, that is, $-\beta \ln U$.

To see why this algorithm works, recall that the (*cumulative*) *distribution function* of a random variable X is defined, for any real x, to be $F(x) = P(X \leq x)$

(Chap. 4 contains a review of basic probability theory). If X is exponential with mean β, then

$$\begin{aligned} F(x) &= \int_0^x \frac{1}{\beta} e^{-t/\beta}\, dt \\ &= 1 - e^{-x/\beta} \end{aligned}$$

for any real $x \geq 0$, since the probability density function of the exponential distribution at the argument $t \geq 0$ is $(1/\beta)e^{-t/\beta}$. To show that our method is correct, we can try to verify that the value it returns will be less than or equal to x (any nonnegative real number), with probability $F(x)$ given above:

$$\begin{aligned} P(-\beta \ln U \leq x) &= P\left(\ln U \geq -\frac{x}{\beta}\right) \\ &= P(U \geq e^{-x/\beta}) \\ &= P(e^{-x/\beta} \leq U \leq 1) \\ &= 1 - e^{-x/\beta} \end{aligned}$$

The first line in the above is obtained by dividing through by $-\beta$ (recall that $\beta > 0$, so $-\beta < 0$ and the inequality reverses), the second line is obtained by exponentiating both sides (the exponential function is monotone increasing, so the inequality is preserved), the third line is just rewriting, together with knowing that U is in [0, 1] anyway, and the last line follows since U is U(0, 1), and the interval $[e^{-x/\beta}, 1]$ is contained within the interval [0, 1]. Since the last line is $F(x)$ for the exponential distribution, we have verified that our algorithm is correct. Chapter 8 discusses how to generate random variates and processes in general.

In our program, we will use a particular method for random-number generation to obtain the variate U described above, as expressed in the C code of Figs. 7.5 and 7.6 in App. 7A of Chap. 7. While most compilers do have some kind of built-in random-number generator, many of these are of extremely poor quality and should not be used; this issue is discussed fully in Chap. 7.

It is convenient (if not the most computationally efficient) to modularize the programs into several subprograms to clarify the logic and interactions, as discussed in general in Sec. 1.3.2. In addition to a main program, the simulation program includes routines for initialization, timing, report generation, and generating exponential random variates, as in Fig. 1.3. It also simplifies matters if we write a separate routine to update the continuous-time statistics, being the accumulated areas under the $Q(t)$ and $B(t)$ curves. The most important action, however, takes place in the routines for the events, which we number as follows:

Event description	Event type
Arrival of a customer to the system	1
Departure of a customer from the system after completing service	2

Figure 1.8 contains a flowchart for the arrival event. First, the time of the next arrival in the future is generated and placed in the event list. Then a check is made

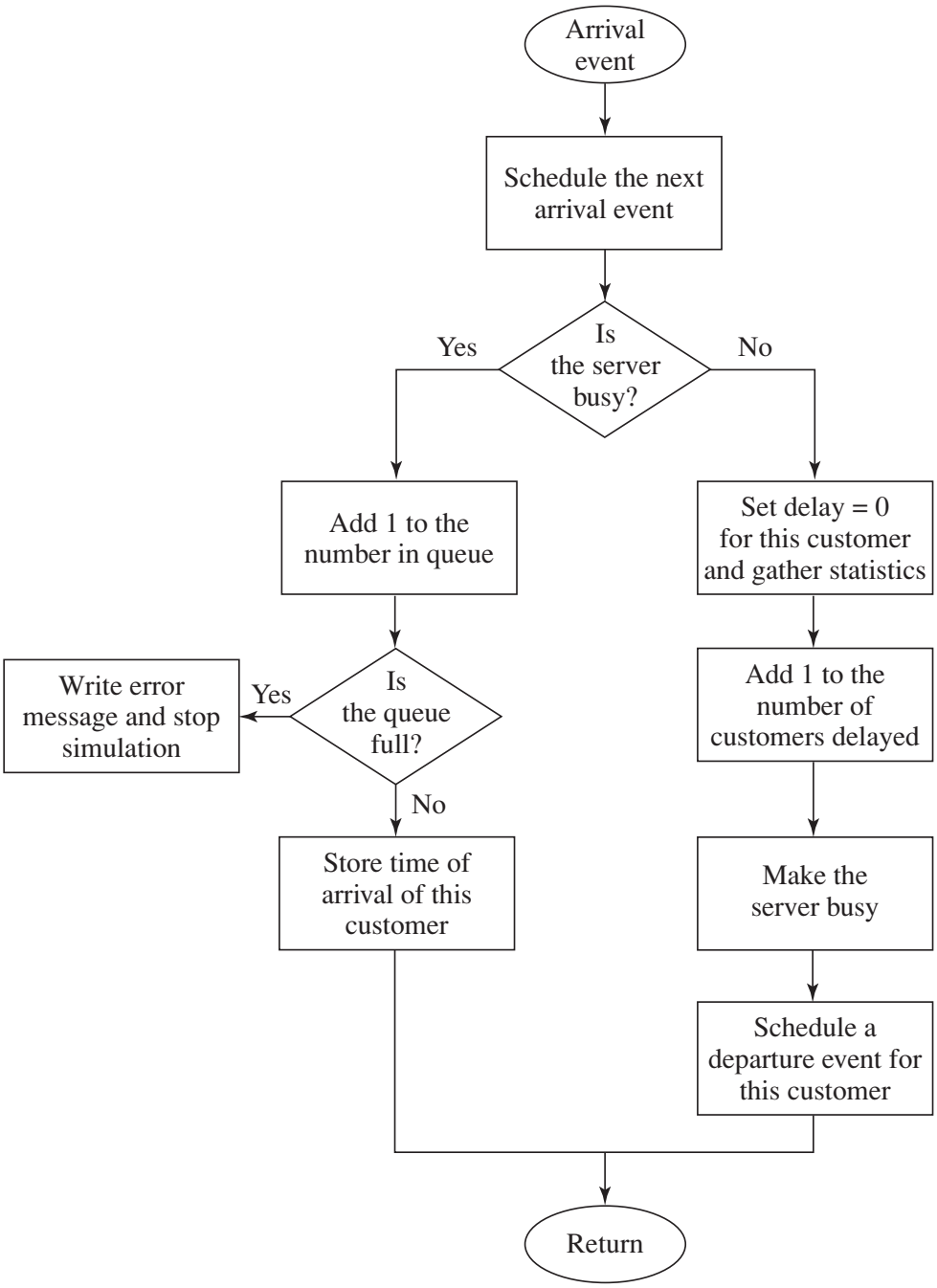


FIGURE 1.8
Flowchart for arrival routine, queueing model.

to determine whether the server is busy. If so, the number of customers in the queue is incremented by 1, and we ask whether the storage space allocated to hold the queue is already full (see the code in Sec. 1.4.4). If the queue is already full, an error message is produced and the simulation is stopped; if there is still room in the queue, the arriving customer's time of arrival is put at the (new) end of the queue. (This queue-full check could be eliminated if using dynamic storage allocation in a programming language that supports this.) On the other hand, if the arriving customer finds the server idle, then this customer has a delay of 0, which *is* counted as a delay, and the number of customer delays completed is incremented by 1. The

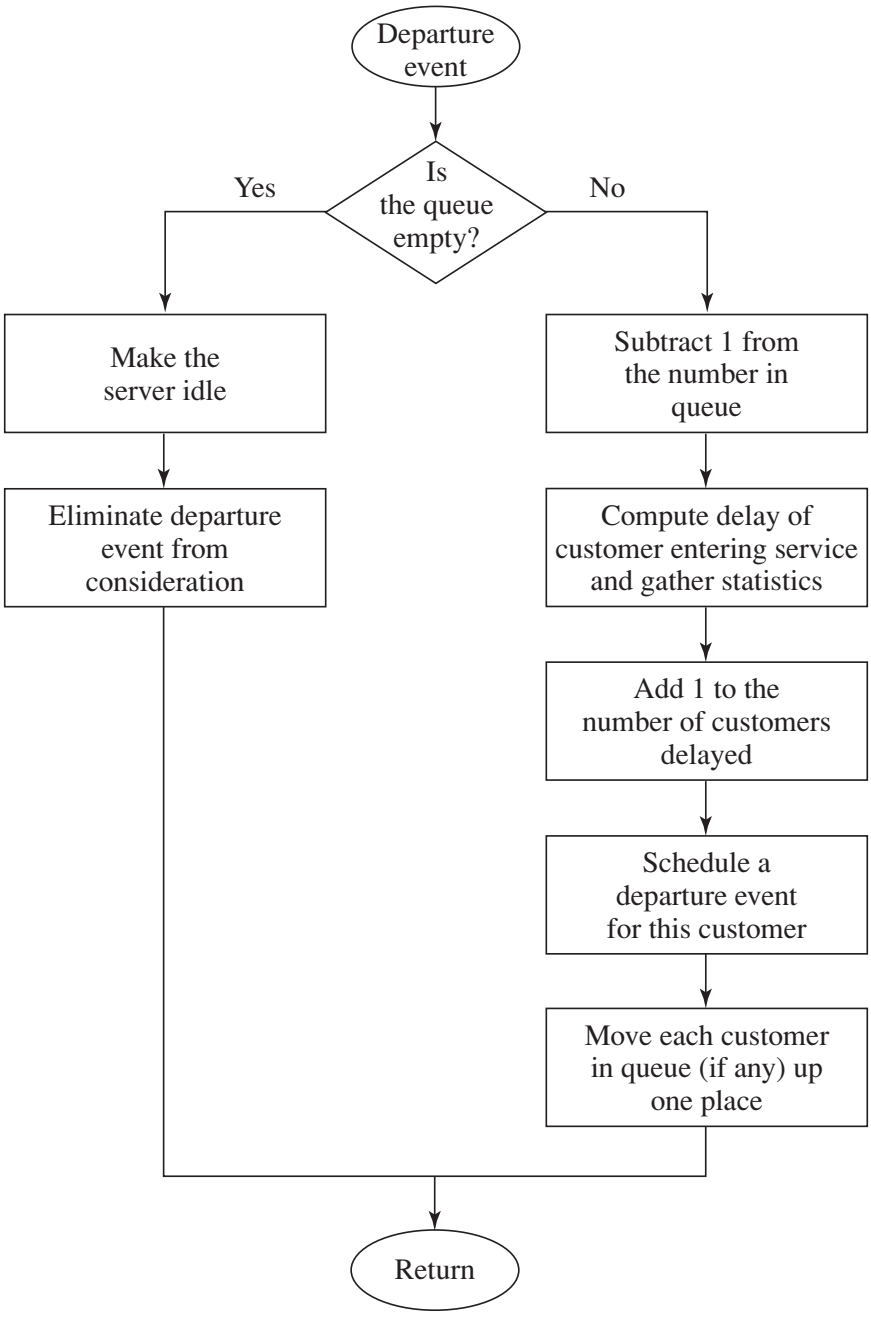


FIGURE 1.9
Flowchart for departure routine, queueing model.

server must be made busy, and the time of departure from service of the arriving customer is scheduled into the event list.

The departure event's logic is depicted in the flowchart of Fig. 1.9. Recall that this routine is invoked when a service completion (and subsequent departure) occurs. If the departing customer leaves no other customers behind in queue, the server is idled and the departure event is eliminated from consideration, since the next event must be an arrival. On the other hand, if one or more customers are left behind by the departing customer, the first customer in queue will leave the queue and enter service, so the queue length is reduced by 1, and the delay in queue of this customer is computed and registered in the appropriate statistical counter. The number

delayed is increased by 1, and a departure event for the customer now entering service is scheduled. Finally, the rest of the queue (if any) is advanced one place. Our implementation of the list for the queue will be very simple in this chapter, and is certainly not the most efficient; Chap. 2 discusses better ways of handling lists to model such things as queues.

In the next section we give an example of how the above setup can be used to write a program in C. The results are discussed in Sec. 1.4.5. This program is neither the simplest nor the most efficient possible, but was instead designed to illustrate how one might organize a program for more complex simulations.

1.4.4 C Program

This section presents a C program for the *M*/*M*/1 queue simulation. We use the ANSI-standard version of the language, as defined by Kernighan and Ritchie (1988), and in particular use function prototyping. We have also taken advantage of C's facility to give variables and functions fairly long names, which thus should be self-explanatory. (For instance, the current value of simulated time is in a variable called sim_time.) We have run our C program on several different computers and compilers. The numerical results differed in some cases due to inaccuracies in floating-point operations. This can matter if, e.g., at some point in the simulation two events are scheduled very close together in time, and roundoff error results in a different sequencing of the event's occurrences. The C math library must be linked, which might require setting an option depending on the compiler. All code is available at www.mhhe.com/law.

The external definitions are given in Fig. 1.10. The header file lcgrand.h (listed in Fig. 7.6) is included to declare the functions for the random-number generator.

```
/* External definitions for single-server queueing system. */

#include <stdio.h>
#include <math.h>
#include "lcgrand.h"  /* Header file for random-number generator. */

#define Q_LIMIT 100  /* Limit on queue length. */
#define BUSY      1  /* Mnemonics for server's being busy */
#define IDLE      0  /* and idle. */

int   next_event_type, num_custs_delayed, num_delays_required, num_events,
      num_in_q, server_status;
float area_num_in_q, area_server_status, mean_interarrival, mean_service,
      sim_time, time_arrival[Q_LIMIT + 1], time_last_event, time_next_event[3],
      total_of_delays;
FILE  *infile, *outfile;

void  initialize(void);
void  timing(void);
void  arrive(void);
void  depart(void);
void  report(void);
void  update_time_avg_stats(void);
float expon(float mean);
```

FIGURE 1.10
C code for the external definitions, queueing model.

The symbolic constant Q_LIMIT is set to 100, our guess (which might have to be adjusted by trial and error) as to the longest the queue will ever get. (As mentioned earlier, this guess could be eliminated if we were using dynamic storage allocation; while C supports this, we have not used it in our examples.) The symbolic constants BUSY and IDLE are defined to be used with the server_status variable, for code readability. File pointers *infile and *outfile are defined to allow us to open the input and output files from within the code, rather than at the operating-system level. Note also that the event list, as we have discussed it so far, will be implemented in an array called time_next_event, whose 0th entry will be ignored in order to make the index agree with the event type.

The code for the main function is shown in Fig. 1.11. The input and output files are opened, and the number of event types for the simulation is initialized to 2 for this model. The input parameters then are read in from the file mm1.in, which contains a single line with the numbers 1.0, 0.5, and 1000, separated by blanks. After writing a report heading and echoing the input parameters (as a check that they were read correctly), the initialization function is invoked. The "while" loop then executes the simulation as long as more customer delays are needed to fulfill the 1000-delay stopping rule. Inside the "while" loop, the timing function is first invoked to determine the type of the next event to occur and to advance the simulation clock to its time. Before processing this event, the function to update the areas under the $Q(t)$ and $B(t)$ curves is invoked; by doing this at this time we automatically update these areas before processing each event. Then a switch statement, based on next_event_type (=1 for an arrival and 2 for a departure), passes control to the appropriate event function. After the "while" loop is done, the report function is invoked, the input and output files are closed, and the simulation ends.

Code for the initialization function is given in Fig. 1.12. Each statement here corresponds to an element of the computer representation in Fig. 1.7*a*. Note that the time of the first arrival, time_next_event[1], is determined by adding an exponential random variate with mean mean_interarrival, namely, expon(mean_interarrival), to the simulation clock, sim_time = 0. (We explicitly used "sim_time" in this statement, although it has a value of 0, to show the general form of a statement to determine the time of a future event.) Since no customers are present at time sim_time = 0, the time of the next departure, time_next_event[2], is set to 1.0e + 30 (C notation for 10^{30}), guaranteeing that the first event will be an arrival.

The timing function, which is given in Fig. 1.13, is used to compare time_next_event[1], time_next_event[2], . . . , time_next_event[num_events] (recall that num_events was set in the main function) and to set next_event_type equal to the event type whose time of occurrence is the smallest. In case of ties, the lowest-numbered event type is chosen. Then the simulation clock is advanced to the time of occurrence of the chosen event type, min_time_next_event. The program is complicated slightly by an error check for the event list's being empty, which we define to mean that all events are scheduled to occur at time = 10^{30}. If this is ever the case (as indicated by next_event_type = 0), an error message is produced along with the current clock time (as a possible debugging aid), and the simulation is terminated.

```
main()  /* Main function. */
{
    /* Open input and output files. */

    infile  = fopen("mm1.in",  "r");
    outfile = fopen("mm1.out", "w");

    /* Specify the number of events for the timing function. */

    num_events = 2;

    /* Read input parameters. */

    fscanf(infile, "%f %f %d", &mean_interarrival, &mean_service,
           &num_delays_required);

    /* Write report heading and input parameters. */

    fprintf(outfile, "Single-server queueing system\n\n");
    fprintf(outfile, "Mean interarrival time%11.3f minutes\n\n",
            mean_interarrival);
    fprintf(outfile, "Mean service time%16.3f minutes\n\n", mean_service);
    fprintf(outfile, "Number of customers%14d\n\n", num_delays_required);

    /* Initialize the simulation. */

    initialize();

    /* Run the simulation while more delays are still needed. */

    while (num_custs_delayed < num_delays_required) {

        /* Determine the next event. */

        timing();

        /* Update time-average statistical accumulators. */

        update_time_avg_stats();

        /* Invoke the appropriate event function. */

        switch (next_event_type) {
            case 1:
                arrive();
                break;
            case 2:
                depart();
                break;
        }
    }

    /* Invoke the report generator and end the simulation. */

    report();

    fclose(infile);
    fclose(outfile);

    return 0;
}
```

FIGURE 1.11
C code for the main function, queueing model.

```
void initialize(void)  /* Initialization function. */
{
    /* Initialize the simulation clock. */

    sim_time = 0.0;

    /* Initialize the state variables. */

    server_status   = IDLE;
    num_in_q        = 0;
    time_last_event = 0.0;

    /* Initialize the statistical counters. */

    num_custs_delayed  = 0;
    total_of_delays    = 0.0;
    area_num_in_q      = 0.0;
    area_server_status = 0.0;

    /* Initialize event list.  Since no customers are present, the departure
       (service completion) event is eliminated from consideration. */

    time_next_event[1] = sim_time + expon(mean_interarrival);
    time_next_event[2] = 1.0e+30;
}
```

FIGURE 1.12
C code for function initialize, queueing model.

```
void timing(void)  /* Timing function. */
{
    int   i;
    float min_time_next_event = 1.0e+29;

    next_event_type = 0;

    /* Determine the event type of the next event to occur. */

    for (i = 1; i <= num_events; ++i)
        if (time_next_event[i] < min_time_next_event) {
            min_time_next_event = time_next_event[i];
            next_event_type     = i;
        }

    /* Check to see whether the event list is empty. */

    if (next_event_type == 0) {

        /* The event list is empty, so stop the simulation. */

        fprintf(outfile, "\nEvent list empty at time %f", sim_time);
        exit(1);
    }

    /* The event list is not empty, so advance the simulation clock. */

    sim_time = min_time_next_event;
}
```

FIGURE 1.13
C code for function timing, queueing model.

```
void arrive(void)  /* Arrival event function. */
{
    float delay;

    /* Schedule next arrival. */

    time_next_event[1] = sim_time + expon(mean_interarrival);

    /* Check to see whether server is busy. */

    if (server_status == BUSY) {

        /* Server is busy, so increment number of customers in queue. */

        ++num_in_q;

        /* Check to see whether an overflow condition exists. */

        if (num_in_q > Q_LIMIT) {

            /* The queue has overflowed, so stop the simulation. */

            fprintf(outfile, "\nOverflow of the array time_arrival at");
            fprintf(outfile, " time %f", sim_time);
            exit(2);
        }

        /* There is still room in the queue, so store the time of arrival of the
           arriving customer at the (new) end of time_arrival. */

        time_arrival[num_in_q] = sim_time;
    }

    else {

        /* Server is idle, so arriving customer has a delay of zero.  (The
           following two statements are for program clarity and do not affect
           the results of the simulation.) */

        delay            = 0.0;
        total_of_delays += delay;

        /* Increment the number of customers delayed, and make server busy. */

        ++num_custs_delayed;
        server_status = BUSY;

        /* Schedule a departure (service completion). */

        time_next_event[2] = sim_time + expon(mean_service);
    }
}
```

FIGURE 1.14
C code for function arrive, queueing model.

The code for event function arrive is in Fig. 1.14, and follows the discussion as given in Sec. 1.4.3 and in the flowchart of Fig. 1.8. Note that "sim_time" is the time of arrival of the customer who is just now arriving, and that the queue-overflow check is made by asking whether num_in_q is now greater than Q_LIMIT, the length for which the array time_arrival was dimensioned.

Event function depart, whose code is shown in Fig. 1.15, is invoked from the main program when a service completion (and subsequent departure) occurs; the

```
void depart(void)  /* Departure event function. */
{
    int   i;
    float delay;

    /* Check to see whether the queue is empty. */

    if (num_in_q == 0) {

        /* The queue is empty so make the server idle and eliminate the
           departure (service completion) event from consideration. */

        server_status      = IDLE;
        time_next_event[2] = 1.0e+30;
    }

    else {

        /* The queue is nonempty, so decrement the number of customers in
           queue. */

        --num_in_q;

        /* Compute the delay of the customer who is beginning service and update
           the total delay accumulator. */

        delay            = sim_time - time_arrival[1];
        total_of_delays += delay;

        /* Increment the number of customers delayed, and schedule departure. */

        ++num_custs_delayed;
        time_next_event[2] = sim_time + expon(mean_service);

        /* Move each customer in queue (if any) up one place. */

        for (i = 1; i <= num_in_q; ++i)
            time_arrival[i] = time_arrival[i + 1];
    }
}
```

FIGURE 1.15
C code for function depart, queueing model.

logic for it was discussed in Sec. 1.4.3, with the flowchart in Fig. 1.9. Note that if the statement "time_next_event[2] = 1.0e + 30;" just before the "else" were omitted, the program would get into an infinite loop. (Why?) Advancing the rest of the queue (if any) one place by the "for" loop near the end of the function ensures that the arrival time of the next customer entering service (after being delayed in queue) will always be stored in time_arrival[1]. Note that if the queue were now empty (i.e., the customer who just left the queue and entered service had been the only one in queue), then num_in_q would be equal to 0, and this loop would not be executed at all since the beginning value of the loop index, i, starts out at a value (1) that would already exceed its final value (num_in_q = 0). (Managing the queue in this simple way is certainly inefficient, and could be improved by using pointers; we return to this issue in Chap. 2.) A final comment about depart concerns the subtraction of time_arrival[1] from the clock value, sim_time, to obtain the delay in queue. If the simulation is to run for a long period of (simulated) time, both sim_time and time_arrival[1] would become very large numbers in comparison with the

```
void report(void)  /* Report generator function. */
{
    /* Compute and write estimates of desired measures of performance. */

    fprintf(outfile, "\n\nAverage delay in queue%11.3f minutes\n\n",
            total_of_delays / num_custs_delayed);
    fprintf(outfile, "Average number in queue%10.3f\n\n",
            area_num_in_q / sim_time);
    fprintf(outfile, "Server utilization%15.3f\n\n",
            area_server_status / sim_time);
    fprintf(outfile, "Time simulation ended%12.3f minutes", sim_time);
}
```

FIGURE 1.16
C code for function report, queueing model.

difference between them; thus, since they are both stored as floating-point (float) numbers with finite accuracy, there is potentially a serious loss of precision when doing this subtraction. For this reason, it may be necessary to make both sim_time and the time_arrival array of type double if we are to run this simulation out for a long period of time.

The code for the report function, invoked when the "while" loop in the main program is over, is given in Fig. 1.16. The average delay is computed by dividing the total of the delays by the number of customers whose delays were observed, and the time-average number in queue is obtained by dividing the area under $Q(t)$, now updated to the end of the simulation (since the function to update the areas is called from the main program before processing either an arrival or departure, one of which will end the simulation), by the clock value at termination. The server utilization is computed by dividing the area under $B(t)$ by the final clock time, and all three measures are written out directly. We also write out the final clock value itself, to see how long it took to observe the 1000 delays.

Function update_time_avg_stats is shown in Fig. 1.17. This function is invoked just before processing each event (of any type) and updates the areas under the two functions needed for the continuous-time statistics; this routine is separate for

```
void update_time_avg_stats(void)  /* Update area accumulators for time-average
                                     statistics. */
{
    float time_since_last_event;

    /* Compute time since last event, and update last-event-time marker. */

    time_since_last_event = sim_time - time_last_event;
    time_last_event       = sim_time;

    /* Update area under number-in-queue function. */

    area_num_in_q      += num_in_q * time_since_last_event;

    /* Update area under server-busy indicator function. */

    area_server_status += server_status * time_since_last_event;
}
```

FIGURE 1.17
C code for function update_time_avg_stats, queueing model.

```
float expon(float mean)  /* Exponential variate generation function. */
{
    /* Return an exponential random variate with mean "mean". */

    return -mean * log(lcgrand(1));
}
```

FIGURE 1.18
C code for function expon.

coding convenience only, and is *not* an event routine. The time since the last event is first computed, and then the time of the last event is brought up to the current time in order to be ready for the next entry into this function. Then the area under the number-in-queue function is augmented by the area of the rectangle under $Q(t)$ during the interval since the previous event, which is of width time_since_last_event and of height num_in_q; remember, this function is invoked *before* processing an event, and state variables such as num_in_q still have their previous values. The area under $B(t)$ is then augmented by the area of a rectangle of width time_since_last_event and height server_status; this is why it is convenient to define server_status to be either 0 or 1. Note that this function, like depart, contains a subtraction of two floating-point numbers (sim_time − time_last_event), both of which could become quite large relative to their difference if we were to run the simulation for a long time; in this case it might be necessary to declare both sim_time and time_last_event to be of type double.

The function expon, which generates an exponential random variate with mean β = mean (passed into expon), is shown in Fig. 1.18, and follows the algorithm discussed in Sec. 1.4.3. The random-number generator lcgrand, used here with an int argument of 1, is discussed fully in Chap. 7, and is shown specifically in Fig. 7.5. The C predefined function log returns the natural logarithm of its argument.

The program described here must be combined with the random-number-generator code from Fig. 7.5. This could be done by separate compilations, followed by linking the object codes together in an installation-dependent way.

1.4.5 Simulation Output and Discussion

The output (in a file named mm1.out) is shown in Fig. 1.19. In this run, the average delay in queue was 0.430 minute, there was an average of 0.418 customer in the queue, and the server was busy 46 percent of the time. It took 1027.915 simulated minutes to run the simulation to the completion of 1000 delays, which seems reasonable since the expected time between customer arrivals was 1 minute. (It is not a coincidence that the average delay, average number in queue, and utilization are all so close together for this model; see App. 1B.)

Note that these particular numbers in the output were determined, at root, by the numbers the random-number generator happened to come up with this time. If a different random-number generator were used, or if this one were used in another way (with another "seed" or "stream," as discussed in Chap. 7), then different numbers would have been produced in the output. Thus, these numbers are not to be regarded

```
Single-server queueing system

Mean interarrival time       1.000 minutes

Mean service time            0.500 minutes

Number of customers           1000


Average delay in queue       0.430 minutes

Average number in queue      0.418

Server utilization           0.460

Time simulation ended     1027.915 minutes
```

FIGURE 1.19
Output report, queueing model.

as "The Answers," but rather as estimates (and perhaps poor ones) of the expected quantities we want to know about, $d(n)$, $q(n)$, and $u(n)$; the statistical analysis of simulation output data is discussed in Chaps. 9 through 12. Also, the results are functions of the input parameters, in this case the mean interarrival and service times, and the $n = 1000$ stopping rule; they are also affected by the way we initialized the simulation (empty and idle).

In some simulation studies, we might want to estimate *steady-state* characteristics of the model (see Chap. 9), i.e., characteristics of a model after the simulation has been running a very long (in theory, an infinite amount of) time. For the simple *M*/*M*/1 queue we have been considering, it is possible to compute *analytically* the steady-state average delay in queue, the steady-state time-average number in queue, and the steady-state server utilization, all of these measures of performance being 0.5 [see, e.g., Ross (2003, pp. 480–487)]. Thus, if we wanted to determine these steady-state measures, our estimates based on the stopping rule $n = 1000$ delays were not too far off, at least in absolute terms. However, we were somewhat lucky, since $n = 1000$ was chosen arbitrarily! In practice, the choice of a stopping rule that will give good estimates of steady-state measures is quite difficult. To illustrate this point, suppose for the *M/M*/1 queue that the arrival rate of customers were increased from 1 per minute to 1.98 per minute (the mean interarrival time is now 0.505 minute), that the mean service time is unchanged, and that we wish to estimate the steady-state measures from a run of length $n = 1000$ delays, as before. We performed this simulation run and got values for the average delay, average number in queue, and server utilization of 17.404 minutes, 34.831, and 0.997, respectively. Since the true steady-state values of these measures are 49.5 minutes, 98.01, and 0.99 (respectively), it is clear that the stopping rule cannot be chosen arbitrarily. We discuss how to specify the run length for a steady-state simulation in Chap. 9.

The reader may have wondered why we did not estimate the expected average waiting time in the system of a customer, $w(n)$, rather than the expected average delay in queue, $d(n)$, where the waiting time of a customer is defined as the time interval from the instant the customer arrives to the instant the customer completes service and departs. There were two reasons. First, for many queueing systems we

believe that the customer's delay in queue while waiting for other customers to be served is the most troublesome part of the customer's wait in the system. Moreover, if the queue represents part of a manufacturing system where the "customers" are actually parts waiting for service at a machine (the "server"), then the delay in queue represents a loss, whereas the time spent in service is "necessary." Our second reason for focusing on the delay in queue is one of statistical efficiency. The usual estimator of $w(n)$ would be

$$\hat{w}(n) = \frac{\sum_{i=1}^{n} W_i}{n} = \frac{\sum_{i=1}^{n} D_i}{n} + \frac{\sum_{i=1}^{n} S_i}{n} = \hat{d}(n) + \overline{S}(n) \tag{1.7}$$

where $W_i = D_i + S_i$ is the waiting time in the system of the ith customer and $\overline{S}(n)$ is the average of the n customers' service times. Since the service-time distribution would have to be known to perform a simulation in the first place, the expected or mean service time, $E(S)$, would also be known and an alternative estimator of $w(n)$ is

$$\tilde{w}(n) = \hat{d}(n) + E(S)$$

[Note that $\overline{S}(n)$ is an unbiased estimator of $E(S)$ in Eq. (1.7).] In almost all queueing simulations, $\tilde{w}(n)$ will be a more efficient (less variable) estimator of $w(n)$ than $\hat{w}(n)$ and is thus preferable (both estimators are unbiased). Therefore, if one wants an estimate of $w(n)$, estimate $d(n)$ and add the known expected service time, $E(S)$. In general, the moral is to replace estimators by their expected values whenever possible (see the discussion of indirect estimators in Sec. 11.5).

1.4.6 Alternative Stopping Rules

In the above queueing example, the simulation was terminated when the number of customers delayed became equal to 1000; the final value of the simulation clock was thus a random variable. However, for many real-world models, the simulation is to stop after some fixed amount of time, say 8 hours. Since the interarrival and service times for our example are continuous random variables, the probability of the simulation's terminating after exactly 480 minutes is 0 (neglecting the finite accuracy of a computer). Therefore, to stop the simulation at a specified time, we introduce a dummy "end-simulation" event (call it an event of type 3), which is scheduled to occur at time 480. When the time of occurrence of this event (being held in the third spot of the event list) is less than all other entries in the event list, the report generator is called and the simulation is terminated. The number of customers delayed is now a random variable.

These ideas can be implemented in the program by making changes in the external definitions, the main function, and the initialize and report functions, as shown in Figs. 1.20 through 1.23; the rest of the program is unaltered. In Figs. 1.20 and 1.21, note that we now have three events; that the desired simulation run length, time_end, is now an input parameter (num_delays_required has been removed); and

```
/* External definitions for single-server queueing system, fixed run length. */

#include <stdio.h>
#include <math.h>
#include "lcgrand.h"  /* Header file for random-number generator. */

#define Q_LIMIT 100  /* Limit on queue length. */
#define BUSY      1  /* Mnemonics for server's being busy */
#define IDLE      0  /* and idle. */

int   next_event_type, num_custs_delayed, num_events, num_in_q, server_status;
float area_num_in_q, area_server_status, mean_interarrival, mean_service,
      sim_time, time_arrival[Q_LIMIT + 1], time_end, time_last_event,
      time_next_event[4], total_of_delays;
FILE  *infile, *outfile;

void  initialize(void);
void  timing(void);
void  arrive(void);
void  depart(void);
void  report(void);
void  update_time_avg_stats(void);
float expon(float mean);
```

FIGURE 1.20
C code for the external definitions, queueing model with fixed run length.

that the "switch" statement has been changed. To stop the simulation, the original "while" loop has been replaced by a "do while" loop in Fig. 1.21, where the loop keeps repeating itself as long as the type of event just executed is not 3 (end simulation); after a type 3 event is chosen for execution, the loop ends and the simulation stops. In the main program (as before), we invoke update_time_avg_stats before entering an event function, so that in particular the areas will be updated to the end of the simulation here when the type 3 event (end simulation) is next. The only change to the initialization function in Fig. 1.22 is the addition of the statement time_next_event[3] = time_end, which schedules the end of the simulation. The only change to the report function in Fig. 1.23 is to write the number of customers delayed instead of the time the simulation ends, since in this case we know that the ending time will be 480 minutes but will not know how many customer delays will have been completed during that time.

The output file (named mm1alt.out) is shown in Fig. 1.24. The number of customer delays completed was 475 in this run, which seems reasonable in a 480-minute run where customers are arriving at an average rate of 1 per minute. The same three measures of performance are again numerically close to each other, but the first two are somewhat less than their earlier values in the 1000-delay simulation. A possible reason for this is that the current run is roughly only half as long as the earlier one, and since the initial conditions for the simulation are empty and idle (an uncongested state), the model in this shorter run has less chance to become congested. Again, however, this is just a single run and is thus subject to perhaps considerable uncertainty; there is no easy way to assess the degree of uncertainty from only a single run.

If the queueing system being considered had actually been a one-operator barbershop open from 9 A.M. to 5 P.M., stopping the simulation after exactly 8 hours

```
main()  /* Main function. */
{
    /* Open input and output files. */

    infile  = fopen("mm1alt.in",  "r");
    outfile = fopen("mm1alt.out", "w");

    /* Specify the number of events for the timing function. */

    num_events = 3;

    /* Read input parameters. */

    fscanf(infile, "%f %f %f", &mean_interarrival, &mean_service, &time_end);

    /* Write report heading and input parameters. */

    fprintf(outfile, "Single-server queueing system with fixed run");
    fprintf(outfile, " length\n\n");
    fprintf(outfile, "Mean interarrival time%11.3f minutes\n\n",
            mean_interarrival);
    fprintf(outfile, "Mean service time%16.3f minutes\n\n", mean_service);
    fprintf(outfile, "Length of the simulation%9.3f minutes\n\n", time_end);

    /* Initialize the simulation. */

    initialize();

    /* Run the simulation until it terminates after an end-simulation event
       (type 3) occurs. */

    do {

        /* Determine the next event. */

        timing();

        /* Update time-average statistical accumulators. */

        update_time_avg_stats();

        /* Invoke the appropriate event function. */

        switch (next_event_type) {
            case 1:
                arrive();
                break;
            case 2:
                depart();
                break;
            case 3:
                report();
                break;
        }

    /* If the event just executed was not the end-simulation event (type 3),
       continue simulating.  Otherwise, end the simulation. */

    } while (next_event_type != 3);

    fclose(infile);
    fclose(outfile);

    return 0;
}
```

FIGURE 1.21
C code for the main function, queueing model with fixed run length.

```
void initialize(void)  /* Initialization function. */
{
    /* Initialize the simulation clock. */

    sim_time = 0.0;

    /* Initialize the state variables. */

    server_status   = IDLE;
    num_in_q        = 0;
    time_last_event = 0.0;

    /* Initialize the statistical counters. */

    num_custs_delayed  = 0;
    total_of_delays    = 0.0;
    area_num_in_q      = 0.0;
    area_server_status = 0.0;

    /* Initialize event list.  Since no customers are present, the departure
       (service completion) event is eliminated from consideration.  The end-
       simulation event (type 3) is scheduled for time time_end. */

    time_next_event[1] = sim_time + expon(mean_interarrival);
    time_next_event[2] = 1.0e+30;
    time_next_event[3] = time_end;
}
```

FIGURE 1.22
C code for function initialize, queueing model with fixed run length.

```
void report(void)  /* Report generator function. */
{
    /* Compute and write estimates of desired measures of performance. */

    fprintf(outfile, "\n\nAverage delay in queue%11.3f minutes\n\n",
            total_of_delays / num_custs_delayed);
    fprintf(outfile, "Average number in queue%10.3f\n\n",
            area_num_in_q / sim_time);
    fprintf(outfile, "Server utilization%15.3f\n\n",
            area_server_status / sim_time);
    fprintf(outfile, "Number of delays completed%7d",
            num_custs_delayed);
}
```

FIGURE 1.23
C code for function report, queueing model with fixed run length.

```
Single-server queueing system with fixed run length

Mean interarrival time      1.000 minutes

Mean service time           0.500 minutes

Length of the simulation  480.000 minutes


Average delay in queue      0.399 minutes

Average number in queue     0.394

Server utilization          0.464

Number of delays completed    475
```

FIGURE 1.24
Output report, queueing model with fixed run length.

might leave a customer with hair partially cut. In such a case, we might want to close the door of the barbershop after 8 hours but continue to run the simulation until all customers present when the door closes (if any) have been served. The reader is asked in Prob. 1.10 to supply the program changes necessary to implement this stopping rule (see also Sec. 2.6).

1.4.7 Determining the Events and Variables

We defined an event in Sec. 1.3 as an instantaneous occurrence that may change the system state, and in the simple single-server queue of Sec. 1.4.1 it was not too hard to identify the events. However, the question sometimes arises, especially for complex systems, of how one determines the number and definition of events in general for a model. It may also be difficult to specify the state variables needed to keep the simulation running in the correct event sequence and to obtain the desired output measures. There is no completely general way to answer these questions, and different people may come up with different ways of representing a model in terms of events and variables, all of which may be correct. But there are some principles and techniques to help simplify the model's structure and to avoid logical errors.

Schruben (1983b) presented an *event-graph* method, which was subsequently refined and extended by Sargent (1988) and Som and Sargent (1989). In this approach proposed events, each represented by a *node*, are connected by *directed arcs* (arrows) depicting how events may be scheduled from other events and from themselves. For example, in the queueing simulation of Sec. 1.4.3, the arrival event schedules another future occurrence of itself and (possibly) a departure event, and the departure event may schedule another future occurrence of itself; in addition, the arrival event must be initially scheduled in order to get the simulation going. Event graphs connect the proposed set of events (nodes) by arcs indicating the type of event scheduling that can occur. In Fig. 1.25 we show the event graph for our single-server queueing system, where the heavy, smooth arrows indicate that an event at the end of the arrow *may* be scheduled from the event at the beginning of the arrow in a (possibly) *nonzero* amount of time, and the thin jagged arrow indicates that the event at its end is scheduled initially. Thus, the arrival event reschedules itself and may schedule a departure (in the case of an arrival who finds the server idle), and the departure event may reschedule itself (if a departure leaves behind someone else in queue).

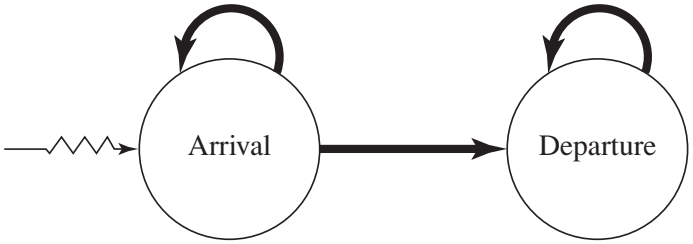


FIGURE 1.25
Event graph, queueing model.

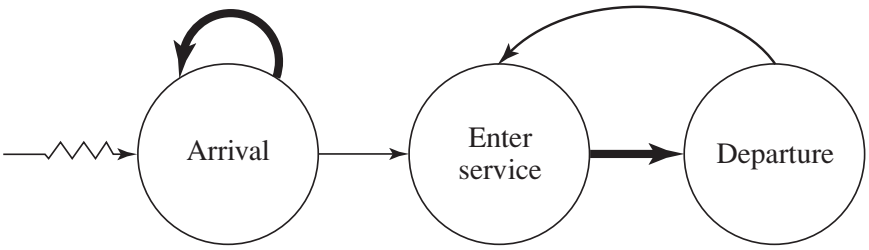


FIGURE 1.26
Event graph, queueing model with separate "enter-service" event.

For this model, it could be asked why we did not explicitly account for the act of a customer's entering service (either from the queue or upon arrival) as a separate event. This certainly can happen, and it could cause the state to change (i.e., the queue length to fall by 1). In fact, this could have been put in as a separate event without making the simulation incorrect, and would give rise to the event diagram in Fig. 1.26. The two thin smooth arrows each represent an event at the beginning of an arrow potentially scheduling an event at the end of the arrow without any intervening time, i.e., immediately; in this case the straight thin smooth arrow refers to a customer who arrives to an empty system and whose "enter-service" event is thus scheduled to occur immediately, and the curved thin smooth arrow represents a customer departing with a queue left behind, and so the first customer in the queue would be scheduled to enter service immediately. The number of events has now increased by 1, and so we have a somewhat more complicated representation of our model. One of the uses of event graphs is to simplify a simulation's event structure by eliminating unnecessary events. There are several "rules" that allow for simplification, and one of them is that if an event node has incoming arcs that are all thin and smooth (i.e., the only way this event is scheduled is by other events and without any intervening time), then this event can be eliminated from the model and its action built into the events that schedule it in zero time. Here, the "enter-service" event could be eliminated, and its action put partly into the arrival event (when a customer arrives to an idle server and begins service immediately) and partly into the departure event (when a customer finishes service and there is a queue from which the next customer is taken to enter service); this takes us back to the simpler event graph in Fig. 1.25. Basically, "events" that can happen only in conjunction with other events do not need to be in the model. Reducing the number of events not only simplifies model conceptualization, but may also speed its execution. Care must be taken, however, when "collapsing" events in this way to handle priorities and time ties appropriately.

Another rule has to do with initialization. The event graph is decomposed into *strongly connected* components, within each of which it is possible to "travel" from every node to every other node by following the arcs in their indicated directions. The graph in Fig. 1.25 decomposes into two strongly connected components (with a single node in each), and that in Fig. 1.26 has two strongly connected components (one of which is the arrival node by itself, and the other of which consists of the enter-service and departure nodes). The initialization rule states that in any strongly

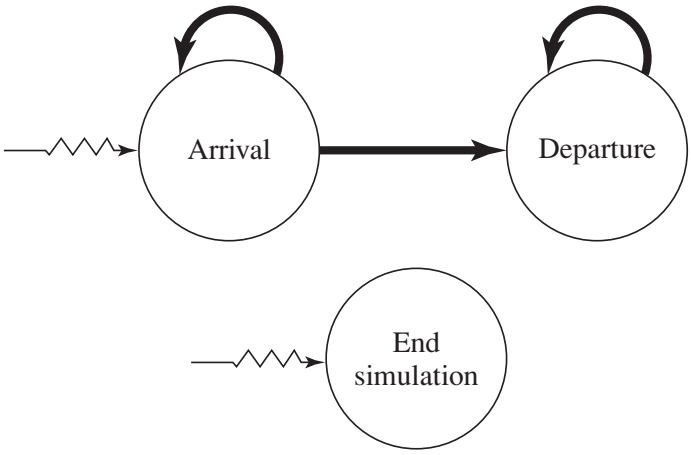


FIGURE 1.27
Event graph, queueing model with fixed run length.

connected component of nodes that has no incoming arcs from other event nodes outside the component, there must be at least one node that is initially scheduled; if this rule were violated, it would never be possible to execute any of the events in the component. In Figs. 1.25 and 1.26, the arrival node is such a strongly connected component since it has no incoming arcs from other nodes, and so it must be initialized. Figure 1.27 shows the event graph for the queueing model of Sec. 1.4.6 with the fixed run length, for which we introduced the dummy "end-simulation" event. Note that this event is itself a strongly connected component without any arcs coming in, and so it must be initialized; i.e., the end of the simulation is scheduled as part of the initialization. Failure to do so would result in erroneous termination of the simulation.

We have presented only a partial and simplified account of the event-graph technique. There are several other features, including event-canceling relations, ways to combine similar events into one, refining the event-scheduling arcs to include conditional scheduling, and incorporating the state variables needed; see the original paper by Schruben (1983b). Sargent (1988) and Som and Sargent (1989) extend and refine the technique, giving comprehensive illustrations involving a flexible manufacturing system and computer network models. Event graphs can also be used to test whether two apparently different models might in fact be equivalent [Yücesan and Schruben (1992)], as well as to forecast how computationally intensive a model will be when it is executed [Yücesan and Schruben (1998)]. Schruben and Schruben (www.sigmawiki.com) developed a software package, SIGMA, for interactive event-graph modeling that runs a model and generates source code. A general event-graph review and tutorial are given by Buss (1996), and advanced applications of event graphs are described in Schruben et al. (2003).

In modeling a system, the event-graph technique can be used to simplify the structure and to detect certain kinds of errors, and is especially useful in complex models involving a large number of interrelated events. Other considerations should also be kept in mind, such as continually asking why a particular state variable is needed; see Prob. 1.4.

1.5 SIMULATION OF AN INVENTORY SYSTEM

We shall now see how simulation can be used to compare alternative ordering policies for an inventory system. Many of the elements of our model are representative of those found in actual inventory systems.

1.5.1 Problem Statement

A company that sells a single product would like to decide how many items it should have in inventory for each of the next n months (n is a fixed input parameter). The times between demands are IID exponential random variables with a mean of 0.1 month. The sizes of the demands, D, are IID random variables (independent of when the demands occur), with

$$D = \begin{cases} 1 & \text{w.p. } \frac{1}{6} \\ 2 & \text{w.p. } \frac{1}{3} \\ 3 & \text{w.p. } \frac{1}{3} \\ 4 & \text{w.p. } \frac{1}{6} \end{cases}$$

where w.p. is read "with probability."

At the beginning of each month, the company reviews the inventory level and decides how many items to order from its supplier. If the company orders Z items, it incurs a cost of $K + iZ$, where $K = \$32$ is the *setup cost* and $i = \$3$ is the *incremental cost* per item ordered. (If $Z = 0$, no cost is incurred.) When an order is placed, the time required for it to arrive (called the *delivery lag* or *lead time*) is a random variable that is distributed uniformly between 0.5 and 1 month.

The company uses a stationary (s, S) policy to decide how much to order, i.e.,

$$Z = \begin{cases} S - I & \text{if } I < s \\ 0 & \text{if } I \geq s \end{cases}$$

where I is the inventory level at the beginning of the month.

When a demand occurs, it is satisfied immediately if the inventory level is at least as large as the demand. If the demand exceeds the inventory level, the excess of demand over supply is backlogged and satisfied by future deliveries. (In this case, the new inventory level is equal to the old inventory level minus the demand size, resulting in a negative inventory level.) When an order arrives, it is first used to eliminate as much of the backlog (if any) as possible; the remainder of the order (if any) is added to the inventory.

So far, we have discussed only one type of cost incurred by the inventory system, the ordering cost. However, most real inventory systems also have two additional types of costs, *holding* and *shortage* costs, which we discuss after introducing some additional notation. Let $I(t)$ be the inventory level at time t [note that $I(t)$ could be positive, negative, or zero]; let $I^+(t) = \max\{I(t), 0\}$ be the number of items physically on hand in the inventory at time t [note that $I^+(t) \geq 0$]; and

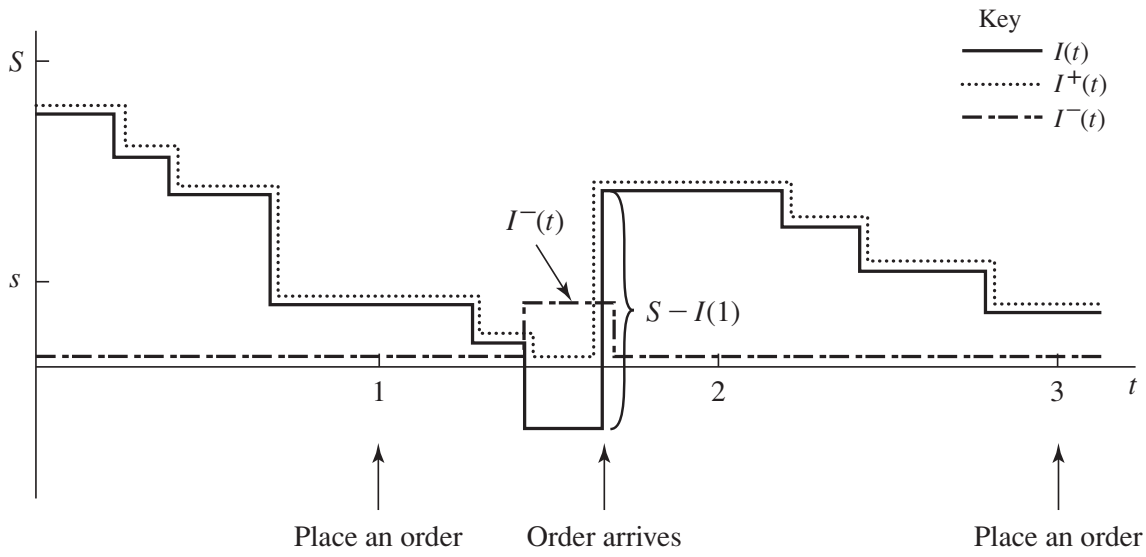


FIGURE 1.28
A realization of $I(t)$, $I^+(t)$, and $I^-(t)$ over time.

let $I^-(t) = \max\{-I(t), 0\}$ be the backlog at time t [$I^-(t) \geq 0$ as well]. A possible realization of $I(t)$, $I^+(t)$, and $I^-(t)$ is shown in Fig. 1.28. The time points at which $I(t)$ decreases are the ones at which demands occur.

For our model, we shall assume that the company incurs a holding cost of $h = \$1$ per item per month held in (positive) inventory. The holding cost includes such costs as warehouse rental, insurance, taxes, and maintenance, as well as the opportunity cost of having capital tied up in inventory rather than invested elsewhere. We have ignored in our formulation the fact that some holding costs are still incurred when $I^+(t) = 0$. However, since our goal is to *compare* ordering policies, ignoring this factor, which after all is independent of the policy used, will not affect our assessment of which policy is best. Now, since $I^+(t)$ is the number of items held in inventory at time t, the time-average (per month) number of items held in inventory for the n-month period is

$$\bar{I}^+ = \frac{\int_0^n I^+(t)\, dt}{n}$$

which is akin to the definition of the time-average number of customers in queue given in Sec. 1.4.1. Thus, the average holding cost per month is $h\bar{I}^+$.

Similarly, suppose that the company incurs a backlog cost of $\pi = \$5$ per item per month in backlog; this accounts for the cost of extra record keeping when a backlog exists, as well as loss of customers' goodwill. The time-average number of items in backlog is

$$\bar{I}^- = \frac{\int_0^n I^-(t)\, dt}{n}$$

so the average backlog cost per month is $\pi\bar{I}^-$.

Assume that the initial inventory level is $I(0) = 60$ and that no order is outstanding. We simulate the inventory system for $n = 120$ months and use the average total cost per month (which is the sum of the average ordering cost per month, the average holding cost per month, and the average shortage cost per month) to compare the following nine inventory policies:

s	20	20	20	20	40	40	40	60	60
S	40	60	80	100	60	80	100	80	100

We do not address here the issue of how these particular policies were chosen for consideration; statistical techniques for making such a determination are discussed in Chap. 12.

Note that the state variables for a simulation model of this inventory system are the inventory level $I(t)$, the amount of an outstanding order from the company to the supplier, and the time of the last event [which is needed to compute the areas under the $I^+(t)$ and $I^-(t)$ functions].

1.5.2 Program Organization and Logic

Our model of the inventory system uses the following types of events:

Event description	Event type
Arrival of an order to the company from the supplier	1
Demand for the product from a customer	2
End of the simulation after n months	3
Inventory evaluation (and possible ordering) at the beginning of a month	4

We have chosen to make the end of the simulation event type 3 rather than type 4, since at time 120 both "end-simulation" and "inventory-evaluation" events will eventually be scheduled and we would like to execute the former event first at this time. (Since the simulation is over at time 120, there is no sense in evaluating the inventory and possibly ordering, incurring an ordering cost for an order that will never arrive.) The execution of event type 3 before event type 4 is guaranteed because the timing routine gives preference to the lowest-numbered event if two or more events are scheduled to occur at the same time. In general, a simulation model should be designed to process events in an appropriate order when time ties occur. An event graph (see Sec. 1.4.7) appears in Fig. 1.29.

There are three types of random variates needed to simulate this system. The interdemand times are distributed exponentially, so the same algorithm (and code) as developed in Sec. 1.4 can be used here. The demand-size random variate D must be discrete, as described above, and can be generated as follows. First divide the unit interval into the contiguous subintervals $C_1 = \left[0, \frac{1}{6}\right)$, $C_2 = \left[\frac{1}{6}, \frac{1}{2}\right)$, $C_3 = \left[\frac{1}{2}, \frac{5}{6}\right)$, and $C_4 = \left[\frac{5}{6}, 1\right]$, and obtain a U(0, 1) random variate U from the random-number

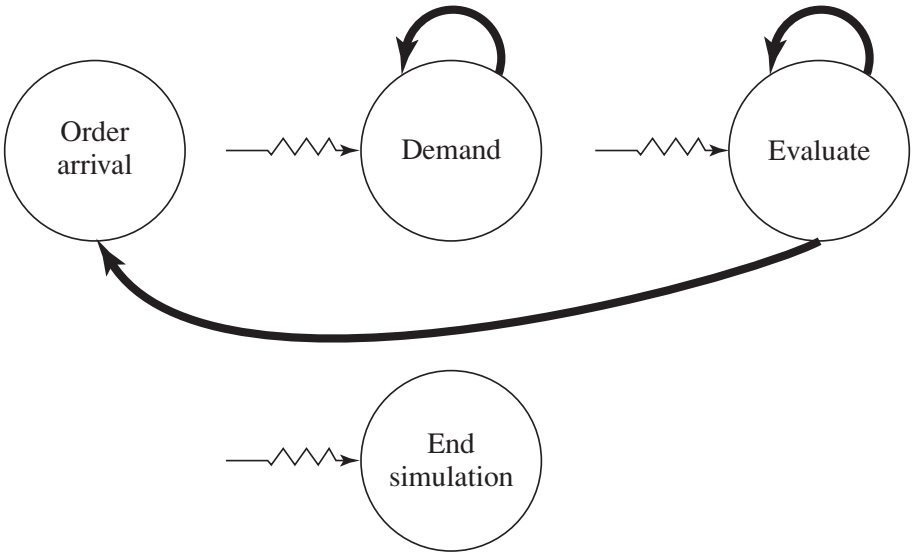


FIGURE 1.29
Event graph, inventory model.

generator. If U falls in C_1, return $D = 1$; if U falls in C_2, return $D = 2$; and so on. Since the width of C_1 is $\frac{1}{6} - 0 = \frac{1}{6}$, and since U is uniformly distributed over [0, 1], the probability that U falls in C_1 (and thus that we return $D = 1$) is $\frac{1}{6}$; this agrees with the desired probability that $D = 1$. Similarly, we return $D = 2$ if U falls in C_2, having probability equal to the width of C_2, $\frac{1}{2} - \frac{1}{6} = \frac{1}{3}$, as desired; and so on for the other intervals. The subprogram to generate the demand sizes uses this principle and takes as input the cutoff points defining the above subintervals, which are the *cumulative* probabilities of the distribution of D.

The delivery lags are uniformly distributed, but not over the unit interval [0,1]. In general, we can generate a random variate distributed uniformly over any interval [a,b] by generating a U(0, 1) random number U, and then returning $a + U(b - a)$. That this method is correct seems intuitively clear, but will be formally justified in Sec. 8.3.1.

We now describe the logic for event types 1, 2, and 4, which actually involve state changes.

The order-arrival event is flowcharted in Fig. 1.30, and must make the changes necessary when an order (which was previously placed) arrives from the supplier. The inventory level is increased by the amount of the order, and the order-arrival event must be eliminated from consideration. (See Prob. 1.12 for consideration of the issue of whether there could be more than one order outstanding at a time for this model with these parameters.)

A flowchart for the demand event is given in Fig. 1.31, and processes the changes necessary to represent a demand's occurrence. First, the demand size is generated, and the inventory is decremented by this amount. Finally, the time of the next demand is scheduled into the event list. Note that this is the place where the inventory level might become negative.

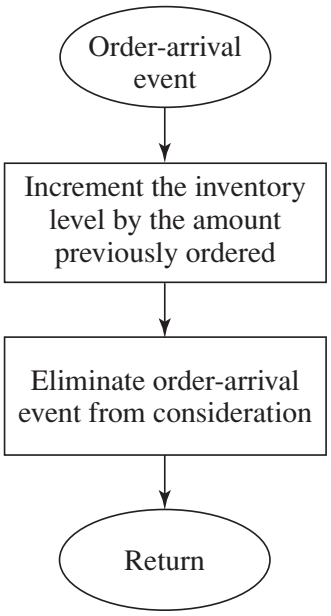


FIGURE 1.30
Flowchart for order-arrival routine, inventory model.

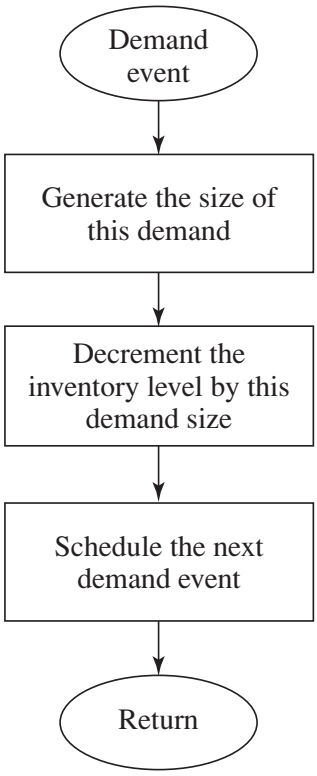


FIGURE 1.31
Flowchart for demand routine, inventory model.

The inventory-evaluation event, which takes place at the beginning of each month, is flowcharted in Fig. 1.32. If the inventory level $I(t)$ at the time of the evaluation is at least s, then no order is placed, and nothing is done except to schedule the next evaluation into the event list. On the other hand, if $I(t) < s$, we want to place an order for $S - I(t)$ items. This is done by storing the amount of the order $[S - I(t)]$ until the order arrives, and scheduling its arrival time. In this case as well, we want to schedule the next inventory-evaluation event.

As in the single-server queueing model, it is convenient to write a separate nonevent routine to update the continuous-time statistical accumulators. For this model, however, doing so is slightly more complicated, so we give a flowchart for this activity in Fig. 1.33. The principal issue is whether we need to update the area under $I^-(t)$ or $I^+(t)$ (or neither). If the inventory level since the last event has been negative, then we have been in backlog, so the area under $I^-(t)$ only should be updated. On the other hand, if the inventory level has been positive, we need only update the area under $I^+(t)$. If the inventory level has been zero (a possibility), then neither update is needed. The code for this routine also brings the variable for the time of the last event up to the present time. This routine will be invoked from the main program just after returning from the timing routine, regardless of the event type or whether the inventory level is actually changing at this point. This provides a simple (if not the most computationally efficient) way of updating integrals for continuous-time statistics.

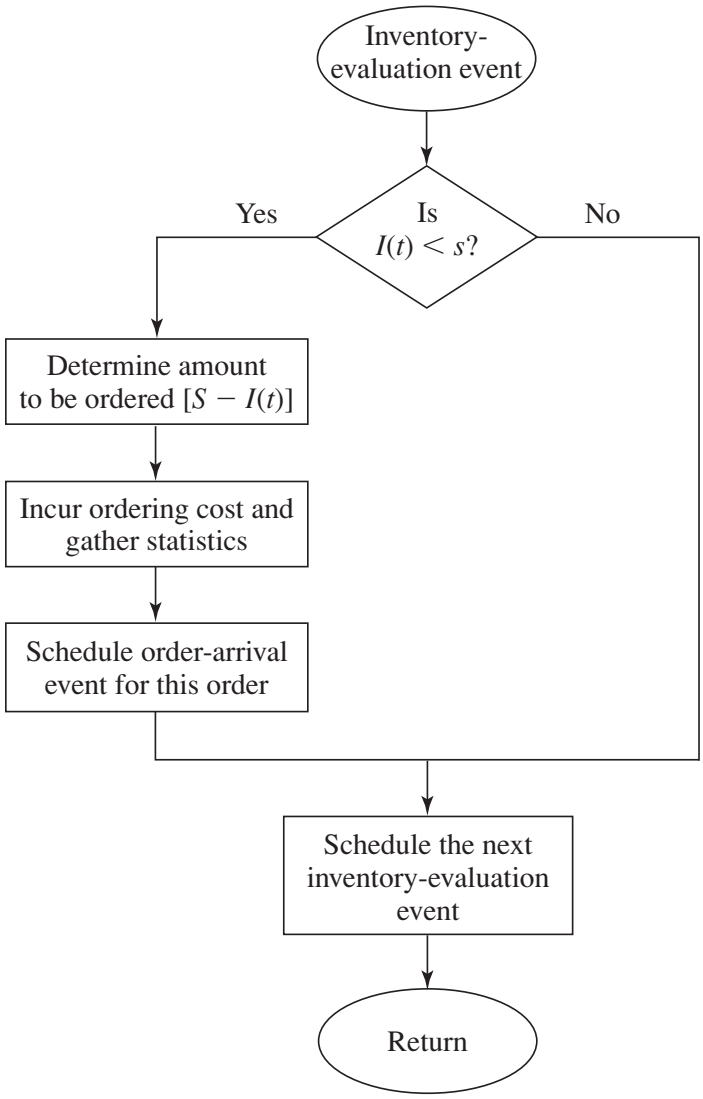


FIGURE 1.32
Flowchart for inventory-evaluation routine, inventory model.

Section 1.5.3 contains a program to simulate this model in C. Neither the timing nor exponential-variate-generation subprograms will be shown, as they are the same as for the single-server queueing model in Sec. 1.4. The reader should also note the considerable similarity between the main programs of the queueing and inventory models.

1.5.3 C Program

The external definitions are shown in Fig. 1.34. The array prob_distrib_demand will be used to hold the cumulative probabilities for the demand sizes, and is passed into the random-integer-generation function random_integer. As for the queueing model, we must include the header file lcgrand.h (in Fig. 7.6) for the random-number generator of Fig. 7.5. All code is available at www.mhhe.com/law.

The code for the main function is given in Fig. 1.35. After opening the input and output files, the number of events is set to 4. The input parameters (except s and S) are then read in and written out, and a report heading is produced; for each (s, S)

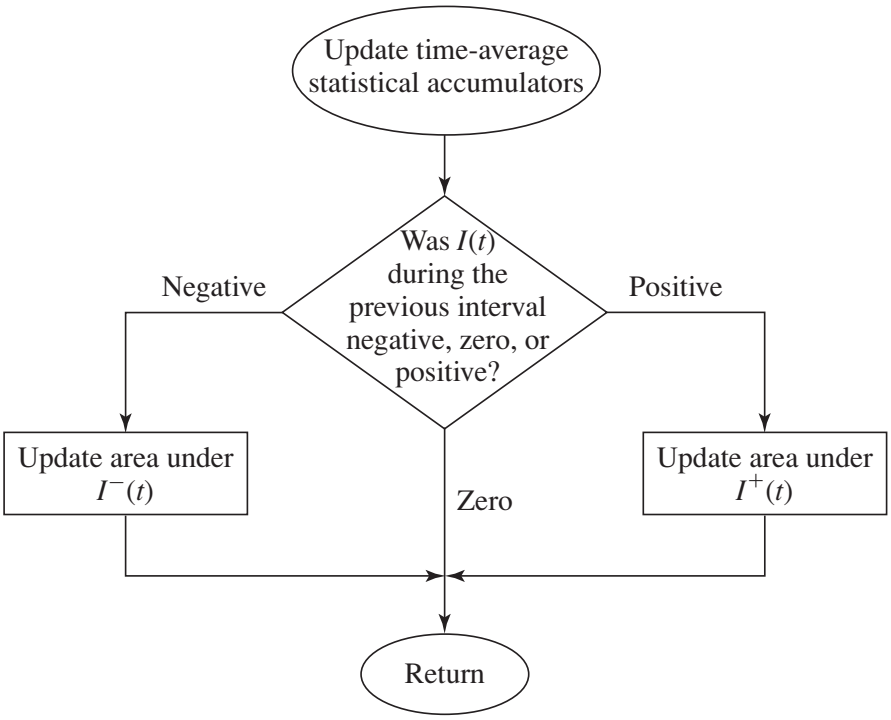


FIGURE 1.33
Flowchart for routine to update the continuous-time statistical accumulators, inventory model.

```
/* External definitions for inventory system. */

#include <stdio.h>
#include <math.h>
#include "lcgrand.h"  /* Header file for random-number generator. */

int   amount, bigs, initial_inv_level, inv_level, next_event_type, num_events,
      num_months, num_values_demand, smalls;
float area_holding, area_shortage, holding_cost, incremental_cost, maxlag,
      mean_interdemand, minlag, prob_distrib_demand[26], setup_cost,
      shortage_cost, sim_time, time_last_event, time_next_event[5],
      total_ordering_cost;
FILE  *infile, *outfile;

void  initialize(void);
void  timing(void);
void  order_arrival(void);
void  demand(void);
void  evaluate(void);
void  report(void);
void  update_time_avg_stats(void);
float expon(float mean);
int   random_integer(float prob_distrib []);
float uniform(float a, float b);
```

FIGURE 1.34
C code for the external definitions, inventory model.

```
main()  /* Main function. */
{
    int i, num_policies;

    /* Open input and output files. */

    infile  = fopen("inv.in",  "r");
    outfile = fopen("inv.out", "w");

    /* Specify the number of events for the timing function. */

    num_events = 4;

    /* Read input parameters. */

    fscanf(infile, "%d %d %d %d %f %f %f %f %f %f %f",
           &initial_inv_level, &num_months, &num_policies, &num_values_demand,
           &mean_interdemand, &setup_cost, &incremental_cost, &holding_cost,
           &shortage_cost, &minlag, &maxlag);
    for (i = 1; i <= num_values_demand; ++i)
        fscanf(infile, "%f", &prob_distrib_demand[i]);

    /* Write report heading and input parameters. */

    fprintf(outfile, "Single-product inventory system\n\n");
    fprintf(outfile, "Initial inventory level%24d items\n\n",
            initial_inv_level);
    fprintf(outfile, "Number of demand sizes%25d\n\n", num_values_demand);
    fprintf(outfile, "Distribution function of demand sizes  ");
    for (i = 1; i <= num_values_demand; ++i)
        fprintf(outfile, "%8.3f", prob_distrib_demand[i]);
    fprintf(outfile, "\n\nMean interdemand time%26.2f\n\n", mean_interdemand);
    fprintf(outfile, "Delivery lag range%29.2f to%10.2f months\n\n", minlag,
            maxlag);
    fprintf(outfile, "Length of the simulation%23d months\n\n", num_months);
    fprintf(outfile, "K =%6.1f   i =%6.1f   h =%6.1f   pi =%6.1f\n\n",
            setup_cost, incremental_cost, holding_cost, shortage_cost);
    fprintf(outfile, "Number of policies%29d\n\n", num_policies);
    fprintf(outfile, "                 Average        Average");
    fprintf(outfile, "        Average        Average\n");
    fprintf(outfile, "  Policy       total cost    ordering cost");
    fprintf(outfile, "  holding cost   shortage cost");

    /* Run the simulation varying the inventory policy. */

    for (i = 1; i <= num_policies; ++i) {

        /* Read the inventory policy, and initialize the simulation. */

        fscanf(infile, "%d %d", &smalls, &bigs);
        initialize();

        /* Run the simulation until it terminates after an end-simulation event
           (type 3) occurs. */

        do {

            /* Determine the next event. */

            timing();

            /* Update time-average statistical accumulators. */

            update_time_avg_stats();

            /* Invoke the appropriate event function. */
```

FIGURE 1.35
C code for the main function, inventory model.

```
            switch (next_event_type) {
                case 1:
                    order_arrival();
                    break;
                case 2:
                    demand();
                    break;
                case 4:
                    evaluate();
                    break;
                case 3:
                    report();
                    break;
            }

        /* If the event just executed was not the end-simulation event (type 3),
           continue simulating.  Otherwise, end the simulation for the current
           (s,S) pair and go on to the next pair (if any). */

        } while (next_event_type != 3);
    }

    /* End the simulations. */

    fclose(infile);
    fclose(outfile);

    return 0;
}
```

FIGURE 1.35
(*continued*)

pair the simulation will then produce in the report function a single line of output corresponding to this heading. Then a "for" loop is begun, each iteration of which performs an entire simulation for a given (*s*, *S*) pair; the first thing done in the loop is to read the next (*s*, *S*) pair. The model is initialized, and a "do while" loop is used to keep simulating as long as the type 3 (end-simulation) event does not occur, as in Sec. 1.4.6. Inside this loop, the timing function is used to determine the next event type and to update the simulation clock. After returning from timing with the next event type, the continuous-time statistics are updated before executing the event routine itself. A "switch" statement is then used as before to transfer control to the appropriate event routine. Unlike the fixed-time stopping rule of Sec. 1.4.6, when the "do while" loop ends here, we do not stop the program, but go to the next step of the enclosing "for" loop to read in the next (*s*, *S*) pair and do a separate simulation; the entire program stops only when the "for" loop is over and there are no more (*s*, *S*) pairs to consider.

The initialization function appears in Fig. 1.36. Observe that the first inventory evaluation is scheduled at time 0 since, in general, the initial inventory level could be less than *s*. Note also that event type 1 (order arrival) is eliminated from consideration, since our modeling assumption was that there are no outstanding orders initially.

The event functions order_arrival, demand, and evaluate are shown in Figs. 1.37 through 1.39, and correspond to the general discussion given in Sec. 1.5.2, and

```
void initialize(void)  /* Initialization function. */
{
    /* Initialize the simulation clock. */

    sim_time = 0.0;

    /* Initialize the state variables. */

    inv_level       = initial_inv_level;
    time_last_event = 0.0;

    /* Initialize the statistical counters. */

    total_ordering_cost = 0.0;
    area_holding        = 0.0;
    area_shortage       = 0.0;

    /* Initialize the event list.  Since no order is outstanding, the order-
       arrival event is eliminated from consideration. */

    time_next_event[1] = 1.0e+30;
    time_next_event[2] = sim_time + expon(mean_interdemand);
    time_next_event[3] = num_months;
    time_next_event[4] = 0.0;
}
```

FIGURE 1.36
C code for function initialize, inventory model.

```
void order_arrival(void)  /* Order arrival event function. */
{
    /* Increment the inventory level by the amount ordered. */

    inv_level += amount;

    /* Since no order is now outstanding, eliminate the order-arrival event from
       consideration. */

    time_next_event[1] = 1.0e+30;
}
```

FIGURE 1.37
C code for function order_arrival, inventory model.

```
void demand(void)  /* Demand event function. */
{
    /* Decrement the inventory level by a generated demand size. */

    inv_level -= random_integer(prob_distrib_demand);

    /* Schedule the time of the next demand. */

    time_next_event[2] = sim_time + expon(mean_interdemand);
}
```

FIGURE 1.38
C code for function demand, inventory model.

```
void evaluate(void)  /* Inventory-evaluation event function. */
{
    /* Check whether the inventory level is less than smalls. */

    if (inv_level < smalls) {

        /* The inventory level is less than smalls, so place an order for the
           appropriate amount. */

        amount               = bigs - inv_level;
        total_ordering_cost += setup_cost + incremental_cost * amount;

        /* Schedule the arrival of the order. */

        time_next_event[1] = sim_time + uniform(minlag, maxlag);
    }

    /* Regardless of the place-order decision, schedule the next inventory
       evaluation. */

    time_next_event[4] = sim_time + 1.0;
}
```

FIGURE 1.39
C code for function evaluate, inventory model.

```
void report(void)  /* Report generator function. */
{
    /* Compute and write estimates of desired measures of performance. */

    float avg_holding_cost, avg_ordering_cost, avg_shortage_cost;

    avg_ordering_cost = total_ordering_cost / num_months;
    avg_holding_cost  = holding_cost * area_holding / num_months;
    avg_shortage_cost = shortage_cost * area_shortage / num_months;
    fprintf(outfile, "\n\n(%3d,%3d)%15.2f%15.2f%15.2f%15.2f",
            smalls, bigs,
            avg_ordering_cost + avg_holding_cost + avg_shortage_cost,
            avg_ordering_cost, avg_holding_cost, avg_shortage_cost);
}
```

FIGURE 1.40
C code for function report, inventory model.

to the flowcharts in Figs. 1.30 through 1.32. In evaluate, note that the variable total_ordering_cost is increased by the ordering cost for any order that might be placed here.

The report generator is listed in Fig. 1.40, and computes the three components of the total cost separately, adding them together to get the average total cost per month. The current values of s and S are written out for identification purposes, along with the average total cost and its three components (ordering, holding, and shortage costs).

Function update_time_avg_stats, which was discussed in general in Sec. 1.5.2 and flowcharted in Fig. 1.33, is shown in Fig. 1.41. Note that if the inventory level inv_level is zero, neither the "if" nor the "else if" condition is satisfied, resulting in no update at all, as desired. As in the single-server queueing model of Sec. 1.4, it

```
void update_time_avg_stats(void)  /* Update area accumulators for time-average
                                     statistics. */
{
    float time_since_last_event;

    /* Compute time since last event, and update last-event-time marker. */

    time_since_last_event = sim_time - time_last_event;
    time_last_event       = sim_time;

    /* Determine the status of the inventory level during the previous interval.
       If the inventory level during the previous interval was negative, update
       area_shortage.  If it was positive, update area_holding.  If it was zero,
       no update is needed. */

    if (inv_level < 0)
        area_shortage -= inv_level * time_since_last_event;
    else if (inv_level > 0)
        area_holding  += inv_level * time_since_last_event;
}
```

FIGURE 1.41
C code for function update_time_avg_stats, inventory model.

```
int random_integer(float prob_distrib[])  /* Random integer generation
                                             function. */
{
    int   i;
    float u;

    /* Generate a U(0,1) random variate. */

    u = lcgrand(1);

    /* Return a random integer in accordance with the (cumulative) distribution
       function prob_distrib. */

    for (i = 1; u >= prob_distrib[i]; ++i)
        ;
    return i;
}
```

FIGURE 1.42
C code for function random_integer.

might be necessary to make both the sim_time and time_last_event variables be of type double to avoid severe roundoff error in their subtraction at the top of the routine if the simulation is to be run for a long period of simulated time.

The code for function random_integer is given in Fig. 1.42, and is general in that it will generate an integer according to distribution function prob_distrib[I], provided that the values of prob_distrib[I] are specified. (In our case, prob_distrib[1] $= \frac{1}{6}$, prob_distrib[2] $= \frac{1}{2}$, prob_distrib[3] $= \frac{5}{6}$, and prob_distrib[4] $= 1$, all specified to three-decimal accuracy on input.) The logic agrees with the discussion in Sec. 1.5.2; note that the input array prob_distrib must contain the *cumulative* distribution function rather than the probabilities that the variate takes on its possible values.

The function uniform is given in Fig. 1.43, and is as described in Sec. 1.5.2.

```
float uniform(float a, float b)  /* Uniform variate generation function. */
{
    /* Return a U(a,b) random variate. */

    return a + lcgrand(1) * (b - a);
}
```

FIGURE 1.43
C code for function uniform.

1.5.4 Simulation Output and Discussion

The simulation report (in file inv.out) is given in Fig. 1.44. For this model, there were some differences in the results across different compilers and computers, even though the same random-number-generator algorithm was being used; see the discussion at the beginning of Sec. 1.4.4 for an explanation of this discrepancy.

```
Single-product inventory system

Initial inventory level                        60 items

Number of demand sizes                          4

Distribution function of demand sizes     0.167   0.500   0.833   1.000

Mean interdemand time                         0.10 months

Delivery lag range                            0.50 to      1.00 months

Length of the simulation                       120 months

K =  32.0   i =   3.0   h =   1.0   pi =   5.0

Number of policies                              9

                Average        Average        Average        Average
  Policy      total cost    ordering cost  holding cost   shortage cost

( 20, 40)         126.61          99.26           9.25          18.10

( 20, 60)         122.74          90.52          17.39          14.83

( 20, 80)         123.86          87.36          26.24          10.26

( 20,100)         125.32          81.37          36.00           7.95

( 40, 60)         126.37          98.43          25.99           1.95

( 40, 80)         125.46          88.40          35.92           1.14

( 40,100)         132.34          84.62          46.42           1.30

( 60, 80)         150.02         105.69          44.02           0.31

( 60,100)         143.20          89.05          53.91           0.24
```

FIGURE 1.44
Output report, inventory model.

The three separate components of the average total cost per month were reported to see how they respond individually to changes in s and S, as a possible check on the model and the code. For example, fixing $s = 20$ and increasing S from 40 to 100 increases the holding cost steadily from \$9.25 per month to \$36.00 per month, while reducing shortage cost at the same time; the effect of this increase in S on the ordering cost is to reduce it, evidently since ordering up to larger values of S implies that these larger orders will be placed less frequently, thereby avoiding the fixed ordering cost more often. Similarly, fixing S at, say, 100, and increasing s from 20 to 60 leads to a decrease in shortage cost (\$7.95, \$1.30, \$0.24) but an increase in holding cost (\$36.00, \$46.42, \$53.91), since increases in s translate into less willingness to let the inventory level fall to low values. While we could probably have predicted the *direction* of movement of these components of cost without doing the simulation, it would not have been possible to say much about their *magnitude* without the aid of the simulation output.

Since the overall criterion of *total* cost per month is the sum of three components that move in sometimes different directions in reaction to changes in s and S, we cannot predict even the direction of movement of this criterion without the simulation. Thus, we simply look at the values of this criterion, and it would *appear* that the (20, 60) policy is the best, having an average total cost of \$122.74 per month. However, in the present context where the length of the simulation is fixed (the company wants a planning horizon of 10 years), what we *really* want to estimate for each policy is the *expected* average total cost per month for the first 120 months. The numbers in Fig. 1.44 are *estimates* of these expected values, each estimate based on a sample of size *1* (simulation run or replication). Since these estimates may have large variances, the ordering of them may differ considerably from the ordering of the expected values, which is the desired information. In fact, if we reran the nine simulations using different U(0, 1) random variates, the estimates obtained might differ greatly from those in Fig. 1.44. Furthermore, the ordering of the new estimates might also be different.

We conclude from the above discussion that when the simulation run length is fixed by the problem context, it will generally not be sufficient to make a single simulation run of each policy or system of interest. In Chap. 9 we address the issue of just how many runs are required to get a good estimate of a desired expected value. Chapters 10 and 12 consider related problems when we are concerned with several different expected values arising from alternative system designs.

1.6 PARALLEL/DISTRIBUTED SIMULATION AND THE HIGH LEVEL ARCHITECTURE

The simulations in Secs. 1.4 and 1.5 (as well as those to be considered in Chap. 2) all operate in basically the same way. A simulation clock and an event list interact to determine which event will be processed next, the simulation clock is advanced to the time of this event, and the computer executes the event logic, which may include updating state variables, updating the event list, and collecting statistics. This

logic is executed in order of the events' simulated time of occurrence; i.e., the simulation is *sequential*. Furthermore, all work is done on a single computer.

In recent years computer technology has enabled individual processors or computers to be linked together into *parallel* or *distributed* computing environments. This allows the possibility of executing different parts of a *single* simulation model on multiple processors operating at the same time, or in "parallel," and thus reducing the overall time to complete the simulation. Alternatively, two or more *different* simulation models operating on separate computers might be tied together over a network to produce one overall simulation model, where the individual models interact with each other over time. In this section we introduce these alternative approaches to executing a simulation model.

1.6.1 Parallel Simulation

Parallel discrete-event simulation [see Fujimoto (1998, 2000, 2003)] is concerned with the execution of a simulation model on a tightly coupled computer system (e.g., a supercomputer or a shared-memory multiprocessor). By spreading the execution of a simulation over several different processors, it is hoped that the model execution time can be reduced considerably (up to a factor equal to the number of processors). For example, if one is simulating a communications network with thousands of nodes or a large military model, then the execution time could be excessive and parallel simulation might be considered. Another possible use for parallel simulation is in real-time decision making. For example, in an air-traffic control system, it might be of interest to simulate several hours of air traffic to decide "now" how best to reroute traffic [see Wieland (1998)].

To develop a parallel simulation, a model is decomposed into several *logical processes* (LPs) (or submodels). The individual LPs (or groups of them) are assigned to different processors, each of which goes to work simulating its piece of the model. The LPs communicate with each other by sending time-stamped messages or events to each other. For example, a manufacturing system is typically modeled as an interconnected network of queueing systems, each representing a different workstation. When a job leaves one workstation, an "arrival" event must be sent to the next station on the job's routing (unless the job is leaving the system).

A crucial issue in parallel simulation is to ensure that events in the *overall* simulation model, regardless of their LP, are processed in their proper time sequence. For example, if the arrival of a particular job to one station is supposed to take place before the departure of another job from a different station, then there must be a synchronization mechanism to make sure that this takes place. If each LP processes all its events (generated either by itself or by another LP) in increasing order of event time, a requirement called the *local causality constraint*, then it can be shown that the parallel simulation will produce exactly the same results as if the overall simulation model were run sequentially on a single computer.

Each LP can be viewed as a sequential discrete-event simulation model, having its own local state variables, event list, and simulation clock. The overall parallel

simulation model, however, does *not* have global counterparts, as would be the case in a sequential simulation model.

Historically, two different types of synchronization mechanisms have been used: conservative and optimistic. In *conservative synchronization* [see Bryant (1977) and Chandy and Misra (1979)], the goal is to absolutely avoid violating the local causality constraint. For example, suppose that a particular LP is currently at simulation time 25 and is ready to process its next event that has an event time of 30. Then the synchronization mechanism must make sure that this LP won't later receive an event from another LP with an event time of less than 30. Thus, the goal is to determine when it is actually "safe" to process a particular event, i.e., when can it be guaranteed that no event will later be received by this LP with a smaller event time.

Conservative synchronization mechanisms have two disadvantages [see Fujimoto (1998, pp. 444–446)]:

1. They cannot fully exploit the parallelism that is available in a simulation application. If event A could possibly affect event B in any way, then A and B must be executed sequentially. If the simulation model is such that A seldom affects B, then A and B could have been processed concurrently most of the time.
2. They are not very robust—a seemingly small change in the model can result in serious degradation in performance.

In *optimistic synchronization*, violations of the local causality constraint are allowed to occur, but the synchronization mechanism detects violations and recovers from them. As above, each LP simulates its own piece of the model forward in time, but does *not* wait to receive messages from other processors that may be moving along at different rates—this waiting is necessary for conservative synchronization.

The *time-warp* mechanism [see Jefferson (1985)] is the best-known optimistic approach. If an LP receives a message that should have been received in its past (and, thus, possibly affecting its actions from that point on), then a *rollback* occurs for the receiving LP, whereby its simulation clock reverts to the (earlier) time of the incoming message. For example, if LP A has been simulated up to time 50 and a message from LP B comes in that should have been received at time 40, then the clock for A is rolled back to 40, and the simulation of A between times 40 and 50 is canceled since it might have been done incorrectly without knowing the contents of the time-40 message. Part of the canceled work may have been sending messages to other LPs, each of which is nullified by sending a corresponding *antimessage*—the antimessages may themselves generate secondary rollbacks at their destination LPs, etc.

Optimistic synchronization mechanisms can exploit the parallelism in a simulation application better than a conservative approach, since they are not limited by the worst-case scenario (see disadvantage 1 for conservative synchronization). However, they do have these disadvantages [see Fujimoto (1998, pp. 449–451)]:

1. They incur the overhead computations associated with executing rollbacks.
2. They require more computer memory since the state of each LP must be saved periodically to recover from a rollback.

The air-traffic-control application described above used an optimistic synchronization mechanism and ran on a four-processor, shared-memory Sun workstation. Another example of optimistic synchronization is the Global-Scale Agent Model (GSAM) of disease propagation [see Parker and Epstein (2011)]. It has been used to simulate a disease outbreak in a population of *6.5 billion* people.

Since computer-processor speeds doubled every 18 months for many years, an increasingly smaller number of simulation models require the use of parallel simulation to execute in a reasonable amount of time. On the other hand, parallel simulation can enable certain models to be run that do not fit into the memory provided by a single machine. For example, Fujimoto et al. (2003) have shown that parallel-simulation methodology allows one to dramatically increase the *size* of communications networks that can be simulated.

1.6.2 Distributed Simulation and the High Level Architecture

Distributed simulation is primarily used to create an overall simulation model, which is a composition of two or more individual simulation models that are located on networked computers. Interest in this form of distributed simulation began with the desire to create real-time, man-in-the-loop simulations that could be used for training military personnel. The SIMNET (SIMulator NETworking) project, which ran from 1983 to 1990, demonstrated the viability of this concept. This led to the creation of a set of protocols for interconnecting simulations, which was known as the Distributed Interactive Simulation (DIS) standard. DIS has given way to the High Level Architecture (HLA) [see Dahmann et al. (1998), Kuhl et al. (2000), and the website www.msco.mil], which was developed by the U.S. Department of Defense (DoD) under the leadership of the Defense Modeling and Simulation Office (DMSO), which has been redesignated as the Modeling & Simulation Coordination Office (MSCO).

The HLA (IEEE Standard 1516-2010) is a software architecture designed to promote the reuse and interoperation of simulations. It was based on the premise that no one simulation could satisfy all uses and applications in the defense industry, and it will ultimately reduce the time and cost required to create a synthetic environment for a new purpose. The HLA can combine the following DoD-defined types of simulations:

- Live—real people operating real systems (e.g., a field test)
- Virtual—real people operating simulated systems (e.g., people in a tank-cockpit simulator fighting simulation-generated enemy forces)
- Constructive—simulated people operating simulated systems (e.g., a discrete-event simulation)

All DoD simulations are supposed to be HLA-compliant beginning January l, 2001 unless a waiver is obtained.

An HLA *federation* consists of a collection of interacting individual simulations, called *federates*, a Runtime Infrastructure (RTI), and an interface, as shown in Fig. 1.45. The RTI provides a set of general-purpose services that support the

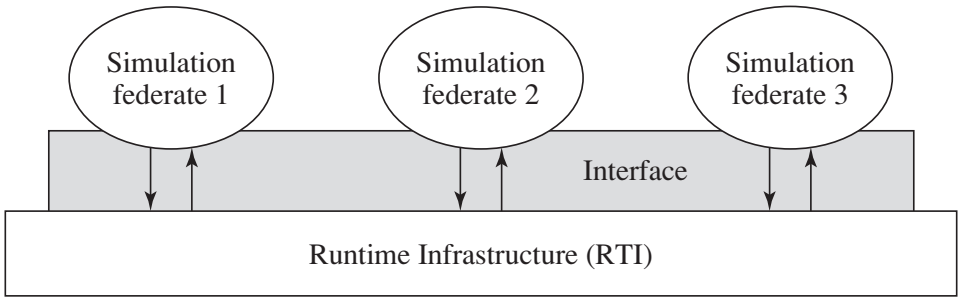


FIGURE 1.45
Functional view of an HLA federation.

simulations in carrying out federate-to-federate interactions, and it also provides functions for federation management. All interactions among the federates go through the RTI, whose software and algorithms are not defined by the HLA. (RTI software can be purchased from third-party vendors.) The HLA runtime interface specification provides a standard mechanism for federates to interact with the RTI, to invoke the RTI services to support interactions among the federates, and to respond to requests from the RTI. The interface is implementation-independent and doesn't depend on the "object" models (e.g., for an entity) and data-exchange requirements of any federate. [HLA objects do not have methods as in classical object-oriented simulation (see Sec. 3.6).]

The HLA is formally defined by three components: the interface specification, the object model template, and the rules. The HLA interface specification describes the runtime services provided to the federates by the RTI and to the RTI by the federates. There are six classes of services that provide capabilities for creation and operation of a federation, for time management (i.e., synchronization), for efficient routing of data among the federates during the execution of the federation, etc.

The HLA object models are descriptions of the essential sharable elements of the federation in object terms. Since the HLA is oriented toward interoperability, object models describe the critical aspects of federates and federations that are shared across the overall simulation model. The HLA puts no constraints on the content of the object models, but does require that these models be documented in a standard format called the Object Model Template (OMT). There are two types of object models: the Federation Object Model (FOM) and the Simulation Object Model (SOM). The HLA FOM describes the set of objects, attributes, and interactions (e.g., an event) that are shared across a federation. The HLA SOM describes a simulation (federate) in terms of the objects, attributes, and interactions that it can offer to future federations, which facilitates the assessment of whether the simulation is appropriate for participation in a new federation.

The HLA rules summarize the key principles underlying the HLA and are divided into two groups: federation and federate rules. Federation rules specify that every federation must have a FOM, that all object representations in a federation reside in the federates rather than in the RTI, etc. Federate rules state that the public information for a simulation is documented in a SOM, that local time management is done using the time-management services provided by the RTI, etc. [See Fujimoto (2003) for a discussion of time management in the HLA.]

HLA federations within the DoD have been used for training military personnel, for test and evaluation of military equipment, and for the analysis of new military systems and tactics. An example of the latter type of application is provided by the U.S. Navy's use of HLA to federate the Network Warfare Simulation (NETWARS) and the Naval Simulation System (NSS) [see Alspaugh et al. (2004) and Murphy and Flournoy (2002)]. This federation uses conservative time management.

There is also interest in distributed simulation and the HLA outside of the defense community. For example, the National Institute of Standards and Technology (NIST) MISSION Project applied HLA to distributed simulations of supply chains involving multiple organizations (e.g., a supplier and a transportation company) [see McLean and Riddick (2000)]. A distributed simulation might be necessary because an organization wants to hide the details of its operations from the other supply-chain organizations.

Although HLA was originally designed to federate two or more individual simulations, it is now also being used to self-federate multiple copies of the *same* simulation model. For example, Fujimoto et al. (2003) and Bodoh and Wieland (2003) use this approach to parallelize simulation models of large-scale communications networks and commercial air-traffic control, respectively.

A totally different use of distributed simulation is to make independent replications of a stand-alone simulation model on networked computers. This will allow an analyst to make more replications of a particular system configuration in a given amount of "wall-clock" time, which will result in more statistically precise estimates of the performance measures of interest. This will also allow an analyst to simulate a larger number of different system configurations in a given amount of wall-clock time when trying to "optimize" the performance of a system of interest (see Sec. 12.5). The simulation packages AutoMod and Simio (see Chap. 3) explicitly support this use of distributed simulation.

Additional information on parallel and distributed simulation can be found in the journal *ACM Transactions on Modeling and Computer Simulation* (TOMACS), as well as in the annual *Proceedings of the Winter Simulation Conference* and the *Proceedings of the Workshop on Principles of Advanced and Distributed Simulation*.

1.7 STEPS IN A SOUND SIMULATION STUDY

Now that we have looked in some detail at the inner workings of a discrete-event simulation, we need to step back and realize that model programming is just part of the overall effort to design or analyze a complex system by simulation. Attention must be paid to a variety of other concerns such as modeling system randomness, validation, statistical analysis of simulation output data, and project management. Figure 1.46 shows the steps that will compose a typical, sound simulation study [see also Banks et al. (2010, pp. 16–21) and Law (2003)]. The number beside the symbol representing each step refers to the more detailed description of that step below. Note that a simulation study is not a simple

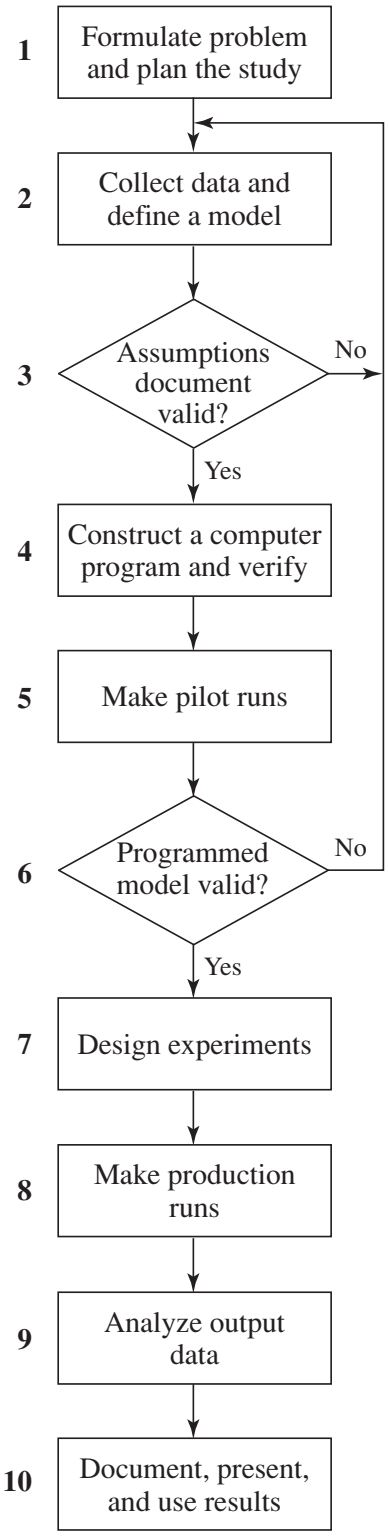


FIGURE 1.46
Steps in a simulation study.

sequential process. As one proceeds with the study, it may be necessary to go back to a previous step.

1. *Formulate the problem and plan the study.*

a. Problem of interest is stated by manager.
- Problem may not be stated correctly or in quantitative terms.
- An iterative process is often necessary.

b. One or more kickoff meetings for the study are conducted, with the project manager, the simulation analysts, and subject-matter experts (SMEs) in attendance. The following things are discussed:
- Overall objectives of the study
- *Specific* questions to be answered by the study (required to decide level of model detail)
- Performance measures that will be used to evaluate the efficacy of different system configurations
- Scope of the model
- System configurations to be modeled (required to decide generality of simulation program)
- Time frame for the study and the required resources

c. Select the software for the model (see Chap. 3)

2. *Collect data and define a model.*

a. Collect information on the system structure and operating procedures.
- No single person or document is sufficient.
- Some people may have inaccurate information—make sure that true SMEs are identified.
- Operating procedures may not be formalized.

b. Collect data (if possible) to specify model parameters and input probability distributions (see Chap. 6).

c. Delineate above information and data in a *written* assumptions document (see Sec. 5.4.3).

d. Collect data (if possible) on the performance of the existing system (for validation purposes in step 6).

e. Choosing the level of model detail (see Sec. 5.2), which is an art, should depend on the following:
- Project objectives
- Performance measures
- Data availability
- Credibility concerns
- Computer constraints
- Opinions of SMEs
- Time and money constraints

f. There should *not* be a one-to-one correspondence between each element of the model and the corresponding element of the system.

g. Start with a "simple" model and embellish it as needed. Modeling each aspect of the system will seldom be required to make effective decisions, and might result in excessive model execution time, in missed deadlines, or in obscuring important system factors.

h. Interact with the manager (and other key project personnel) on a regular basis (see Sec. 5.4.2).

3. *Is the assumptions document valid?*

a. Perform a structured walk-through of the assumptions document before an audience of managers, analysts, and SMEs (see Sec. 5.4.3). This will
- Help ensure that the model's assumptions are correct and complete
- Promote interaction among the project members
- Promote ownership of the model
- Take place *before* programming begins, to avoid significant reprogramming later

4. *Construct a computer program and verify.*

a. Program the model in a programming language (e.g., C, C++, or Java) or in simulation software (e.g., Arena, ExtendSim, Flexsim, and ProModel). Benefits of using a programming language are that one is often known, they offer greater program control, they have a low *purchase* cost, and they may result in a smaller model-execution time. The use of simulation software (see Chap. 3), on the other hand, reduces programming time and results in a lower *project* cost.

b. Verify (debug) the simulation computer program (see Sec. 5.3).

5. *Make pilot runs.*

a. Make pilot runs for validation purposes in step 6.

6. *Is the programmed model valid?*

a. If there is an existing system, then compare model and system (from step 2) performance measures for the existing system (see Sec. 5.4.5).

b. Regardless of whether there is an existing system, the simulation analysts and SMEs should review the model results for correctness.

c. Use sensitivity analyses (see Sec. 5.4.4) to determine what model factors have a significant impact on performance measures and, thus, have to be modeled carefully.

7. *Design experiments.*

a. Specify the following for each system configuration of interest:
- Length of each simulation run
- Length of the warmup period, if one is appropriate
- Number of independent simulation runs using different random numbers (see Chap. 7)—facilitates construction of confidence intervals

8. *Make production runs.*

a. Production runs are made for use in step 9.

9. *Analyze output data.*

a. Two major objectives in analyzing output data are to
- Determine the absolute performance of certain system configurations (see Chap. 9)
- Compare alternative system configurations in a relative sense (see Chap. 10 and Sec. 11.2)

10. *Document, present, and use results.*

a. Document assumptions (see step 2), computer program, and study's results for use in the current and future projects.

b. Present study's results.
- Use animation (see Sec. 3.4.3) to communicate model to managers and other people who are not familiar with all the model details.
- Discuss model building and validation process to promote credibility.
- Results are used in decision-making process if they are *both* valid and credible.

1.8 ADVANTAGES, DISADVANTAGES, AND PITFALLS OF SIMULATION

We conclude this introductory chapter by listing some good and bad characteristics of simulation (as opposed to other methods of studying systems), and by noting some common mistakes made in simulation studies that can impair or even ruin a simulation project. This subject was also discussed to some extent in Sec. 1.2, but now that we have worked through some simulation examples, it is possible to be more specific.

As mentioned in Sec. 1.2, simulation is a widely used and increasingly popular method for studying complex systems. Some possible advantages of simulation that may account for its widespread appeal are the following.

- Most complex, real-world systems with stochastic elements cannot be accurately described by a mathematical model that can be evaluated *analytically*. Thus, a simulation is often the only type of investigation possible.
- Simulation allows one to estimate the performance of an existing system under some projected set of operating conditions.
- Alternative proposed system designs (or alternative operating policies for a single system) can be compared via simulation to see which best meets a specified requirement.
- In a simulation we can maintain much better control over experimental conditions than would generally be possible when experimenting with the system itself (see Chap. 11).
- Simulation allows us to study a system with a long time frame—e.g., an economic system—in compressed time, or alternatively to study the detailed workings of a system in expanded time.

Simulation is not without its drawbacks. Some disadvantages are as follows.

- Each run of a *stochastic* simulation model produces only *estimates* of a model's true characteristics for a particular set of input parameters. Thus, several independent runs of the model will probably be required for each set of input parameters to be studied (see Chap. 9). For this reason, simulation models are generally not as good at optimization as they are at comparing a fixed number of specified alternative system designs. On the other hand, an analytic model, *if appropriate*, can often easily produce the *exact* true characteristics of that model for a variety of sets of input parameters. Thus, if a "valid" analytic model is available or can easily be developed, it will generally be preferable to a simulation model.
- Simulation models are often expensive and time-consuming to develop.
- The large volume of numbers produced by a simulation study or the persuasive impact of a realistic animation (see Sec. 3.4.3) often creates a tendency to place greater confidence in a study's results than is justified. If a model is not a "valid" representation of a system under study, the simulation results, no matter how impressive they appear, will provide little useful information about the actual system.

When deciding whether or not a simulation study is appropriate in a given situation, we can only advise that these advantages and drawbacks be kept in mind and that all other relevant facets of one's particular situation be brought to bear as well. Finally, note that in some studies both simulation and analytic models might be useful. In particular, simulation can be used to check the validity of assumptions needed in an analytic model. On the other hand, an analytic model can suggest reasonable alternatives to investigate in a simulation study.

Assuming that a decision has been made to use simulation, we have found the following pitfalls to the successful completion of a simulation study [see also Law and McComas (1989)]:

- Failure to have a well-defined set of objectives at the beginning of the simulation study
- Failure to have the entire project team involved at the beginning of the study
- Inappropriate level of model detail
- Failure to communicate with management throughout the course of the simulation study
- Misunderstanding of simulation by management
- Treating a simulation study as if it were primarily an exercise in computer programming
- Failure to have people with a knowledge of simulation methodology (Chaps. 5, 6, 9, etc.) and statistics on the modeling team
- Failure to collect good system data
- Inappropriate simulation software
- Obliviously using simulation-software products whose complex macro statements may not be well documented and may not implement the desired modeling logic

- Belief that easy-to-use simulation packages, which require little or no programming, require a significantly lower level of technical competence
- Misuse of animation
- Failure to account correctly for sources of randomness in the actual system
- Using arbitrary distributions (e.g., normal, uniform, or triangular) as input to the simulation
- Analyzing the output data from one simulation run (replication) using formulas that assume independence
- Making a single replication of a particular system design and treating the output statistics as the "true answers"
- Failure to have a warmup period, if the steady-state behavior of a system is of interest
- Comparing alternative system designs on the basis of one replication for each design
- Using the wrong performance measures

We will have more to say about what *to* do (rather than what *not* to do) in the remaining chapters of this book.

APPENDIX 1A
FIXED-INCREMENT TIME ADVANCE

As mentioned in Sec. 1.3.1, the second principal approach for advancing the simulation clock in a discrete-event simulation model is called *fixed-increment time advance*. With this approach, the simulation clock is advanced in increments of exactly Δt time units for some appropriate choice of Δt. After each update of the clock, a check is made to determine if any events should have occurred during the previous interval of length Δt. If one or more events were scheduled to have occurred during this interval, these events are considered to occur at the *end* of the interval and the system state (and statistical counters) are updated accordingly. The fixed-increment time-advance approach is illustrated in Fig. 1.47, where the curved arrows represent the advancing of the simulation clock and e_i ($i = 1, 2, \ldots$) is the *actual* time of occurrence of the ith event of any type (*not* the ith value of the simulation clock). In the time interval $[0, \Delta t)$, an event occurs at time e_1 but is considered to occur at time Δt by the model. No events occur in the interval $[\Delta t, 2\Delta t)$, but the model checks to determine that this is the case. Events occur at the times

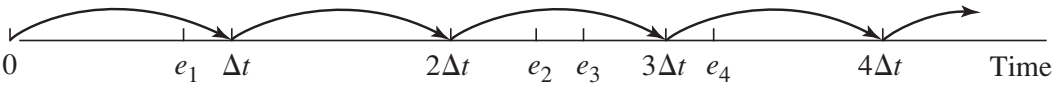


FIGURE 1.47
Illustration of fixed-increment time advance.

e_2 and e_3 in the interval $[2\Delta t, 3\Delta t)$, but both events are considered to occur at time $3\Delta t$, etc. A set of rules must be built into the model to decide in what order to process events when two or more events are considered to occur at the same time by the model. Two disadvantages of fixed-increment time advance are the errors introduced by processing events at the end of the interval in which they occur and the necessity of deciding which event to process first when events that are not simultaneous in reality are treated as such by the model. These problems can be made less severe by making Δt smaller, but this increases the amount of checking for event occurrences that must be done and results in an increase in execution time. Because of these considerations, fixed-increment time advance is generally not used for discrete-event simulation models when the times between successive events can vary greatly.

The primary uses of this approach appear to be for systems where it can reasonably be assumed that all events *actually* occur at one of the times $n\ \Delta t$ ($n = 0, 1, 2, \ldots$) for an appropriately chosen Δt and, unfortunately, for agent-based simulation (see Sec. 13.2). For example, data in economic systems are often available only on an annual basis, and it is natural in a simulation model to advance the simulation clock in increments of 1 year. [See Naylor (1971) for a discussion of simulation of economic systems. See also Sec. 13.6 for discussion of an inventory system that can be simulated, without loss of accuracy, by fixed-increment time advance.]

Note that fixed-increment time advance can be realized when using the next-event time-advance approach by artificially scheduling "events" to occur every Δt time units.

APPENDIX 1B
A PRIMER ON QUEUEING SYSTEMS

A *queueing system* consists of one or more servers that provide service of some kind to arriving customers. Customers who arrive to find all servers busy (generally) join one or more *queues* (or lines) in front of the servers, hence the name "queueing" system.

Historically, a large proportion of all discrete-event simulation studies have involved the modeling of a real-world queueing system, or at least some component of the system being simulated was a queueing system. Thus, we believe that it is important for the student of simulation to have at least a basic understanding of the components of a queueing system, standard notation for queueing systems, and measures of performance that are often used to indicate the quality of service being provided by a queueing system. Some examples of real-world queueing systems that have often been simulated are given in Table 1.1. For additional information on queueing systems in general, see Gross et al. (2009). Bertsekas and Gallager (1992) is recommended for those interested in queueing models of communications networks. Finally, Shanthikumar and Buzacott (1993) discuss stochastic models of manufacturing systems.

TABLE 1.1
Examples of queueing systems

System	Servers	Customers
Bank	Tellers	Customers
Hospital	Doctors, nurses, beds	Patients
Computer system	Central processing unit, input/output devices	Jobs
Manufacturing system	Machines, workers	Parts
Airport	Runways, gates, security check-in stations	Airplanes, travelers
Communications network	Nodes, links	Messages, packets

1B.1 COMPONENTS OF A QUEUEING SYSTEM

A queueing system is characterized by three components: arrival process, service mechanism, and queue discipline. Specifying the *arrival process* for a queueing system consists of describing how customers arrive to the system. Let A_i be the interarrival time between the arrivals of the $(i-1)$st and ith customers (see Sec. 1.3). If $A_1, A_2, \ldots$ are assumed to be IID random variables, we shall denote the *mean* (or expected) *interarrival time* by $E(A)$ and call $\lambda = 1/E(A)$ the *arrival rate* of customers.

The *service mechanism* for a queueing system is articulated by specifying the number of servers (denoted by s), whether each server has its own queue or there is one queue feeding all servers, and the probability distribution of customers' service times. Let S_i be the service time of the ith arriving customer. If $S_1, S_2, \ldots$ are IID random variables, we shall denote the *mean service time* of a customer by $E(S)$ and call $\omega = 1/E(S)$ the *service rate* of a server.

The *queue discipline* of a queueing system refers to the rule that a server uses to choose the next customer from the queue (if any) when the server completes the service of the current customer. Commonly used queue disciplines include

FIFO: Customers are served in a first-in, first-out manner.
LIFO: Customers are served in a last-in, first-out manner (see Prob. 2.17).
Priority: Customers are served in order of their importance (see Prob. 2.22) or on the basis of their service requirements (see Probs. 1.24, 2.20, and 2.21).

1B.2 NOTATION FOR QUEUEING SYSTEMS

Certain queueing systems occur so often in practice that standard notations have been developed for them. In particular, consider the queueing system shown in Fig. 1.48, which has the following characteristics:

1. s servers in parallel and one FIFO queue feeding all servers
2. $A_1, A_2, \ldots$ are IID random variables.
3. $S_1, S_2, \ldots$ are IID random variables.
4. The A_i's and S_i's are independent.

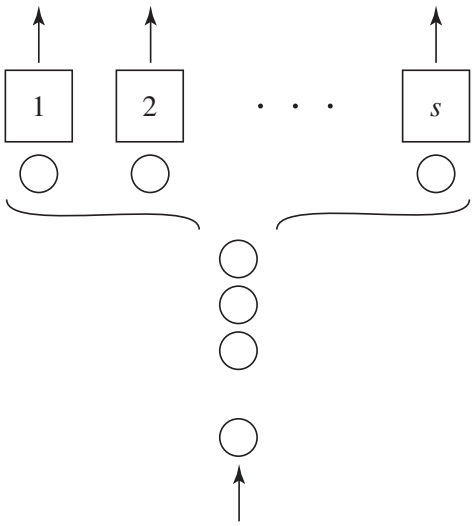


FIGURE 1.48
A *GI*/*G*/*s* queue.

We call such a system a *GI*/*G*/*s* queue, where *GI* (general independent) refers to the distribution of the A_i's and *G* (general) refers to the distribution of the S_i's. If specific distributions are given for the A_i's and the S_i's (as is always the case for simulation), symbols denoting these distributions are used in place of *GI* and *G*. The symbol *M* is used for the exponential distribution because of the Markovian, i.e., memoryless, property of the exponential distribution (see Prob. 4.30), the symbol E_k for a *k*-Erlang distribution (if *X* is a *k*-Erlang random variable, then $X = \sum_{i=1}^{k} Y_i$, where the Y_i's are IID exponential random variables), and *D* for deterministic (or constant) times. Thus, a single-server queueing system with exponential interarrival times and service times and a FIFO queue discipline is called an *M*/*M*/1 queue.

For any *GI*/*G*/*s* queue, we shall call the quantity $\rho = \lambda/(s\omega)$ the *utilization factor* of the queueing system ($s\omega$ is the service rate of the system when all servers are busy). It is a measure of how heavily the resources of a queueing system are utilized.

1B.3 MEASURES OF PERFORMANCE FOR QUEUEING SYSTEMS

There are many possible measures of performance for queueing systems. We now describe four such measures that are usually used in the mathematical study of queueing systems. The reader should not infer from our choices that these measures are necessarily the most relevant or important in practice (see Chap. 9 for further discussion). As a matter of fact, for some real-world systems these measures may not even be well defined; i.e., they may not exist.

Let

D_i = delay in queue of *i*th customer

$W_i = D_i + S_i$ = waiting time in system of *i*th customer

$Q(t)$ = number of customers in queue at time t

$L(t)$ = number of customers in system at time t [$Q(t)$ plus number of customers being served at time t]

Then the measures

$$d = \lim_{n\to\infty} \frac{\sum_{i=1}^{n} D_i}{n} \qquad \text{w.p. 1}$$

and

$$w = \lim_{n\to\infty} \frac{\sum_{i=1}^{n} W_i}{n} \qquad \text{w.p. 1}$$

(if they exist) are called the *steady-state average delay* and the *steady-state average waiting time*. Similarly, the measures

$$Q = \lim_{T\to\infty} \frac{\int_0^T Q(t)\, dt}{T} \qquad \text{w.p. 1}$$

and

$$L = \lim_{T\to\infty} \frac{\int_0^T L(t)\, dt}{T} \qquad \text{w.p. 1}$$

(if they exist) are called the *steady-state time-average number in queue* and the *steady-state time-average number in system*. Here and throughout this book, the qualifier "w.p. 1" (with probability 1) is given for mathematical correctness and has little practical significance. For example, suppose that $\Sigma_{i=1}^{n} D_i/n \to d$ as $n \to \infty$ (w.p. 1) for some queueing system. This means that if one performs a very large (an infinite) number of experiments, then in virtually every experiment $\Sigma_{i=1}^{n} D_i/n$ converges to the finite quantity d. Note that $\rho < 1$ is a necessary condition for d, w, Q, and L to exist for a $GI/G/s$ queue.

Among the most general and useful results for queueing systems are the *conservation equations*

$$Q = \lambda d \qquad \text{and} \qquad L = \lambda w$$

These equations hold for every queueing system for which d and w exist [see Stidham (1974)]. (Section 11.5 gives a simulation application of these relationships.) Another equation of considerable practical value is given by

$$w = d + E(S)$$

(see Sec. 1.4.5 and also Sec. 11.5 for further discussion).

It should be mentioned that the measures of performance discussed above can be analytically computed for $M/M/s$ queues ($s \geq 1$), $M/G/1$ queues for any distribution G, and for certain other queueing systems. In general, the interarrival-time distribution, the service-time distribution, or both must be exponential (or a variant of exponential, such as k-Erlang) for analytic solutions to be possible [see Gross et al. (2009)].

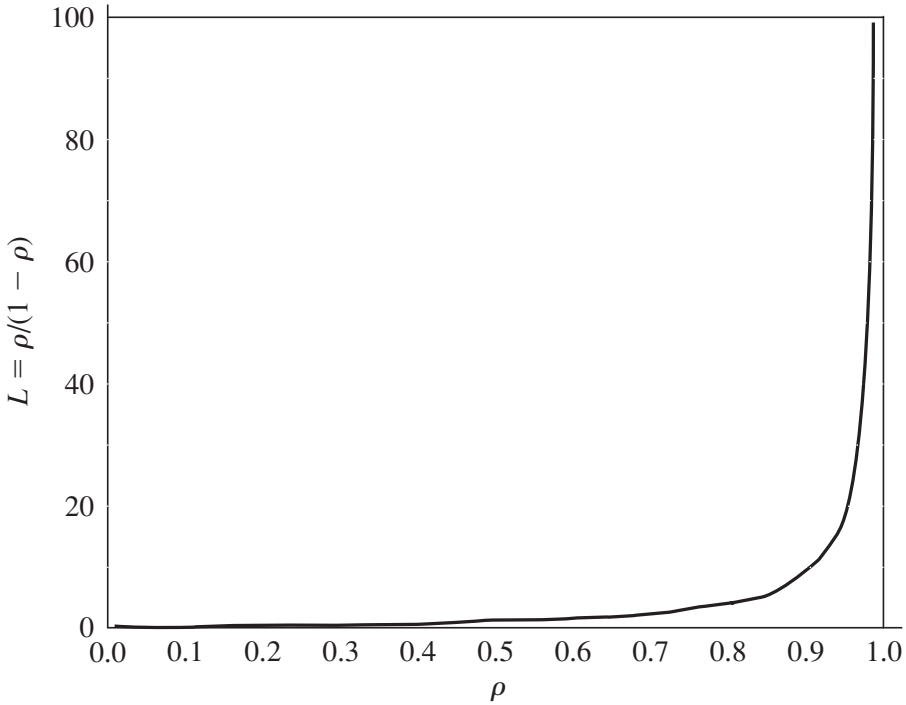


FIGURE 1.49
Plot of $L = \rho/(1 - \rho)$ for the *M*/*M*/1 queue.

For example, in the case of an *M/M/*1 queue, it can be shown analytically that the steady-state average number in system is given by

$$L = \rho/(1 - \rho)$$

which we plot as a function of ρ in Fig. 1.49. Note that L is clearly not a linear function of ρ, and for $\rho > 0.8$ the plot of L increases exponentially. Although the formula for L is specifically for the *M*/*M*/1 queue, the nonlinear behavior seen in Fig. 1.49 is indicative of queueing systems in general.

Another interesting (and instructive) example of an analytical solution is the steady-state average delay in queue for an *M*/*G*/1 queue, given by

$$d = \frac{\lambda\{\text{Var}(S) + [E(S)]^2\}}{2[1 - \lambda E(S)]}$$

where Var(S) denotes the variance of the service-time distribution [see, for example, Ross (2003, p. 508) for a derivation of this formula]. Thus, we can see that if $E(S)$ is large, then congestion (here measured by d) will be larger; this is certainly to be expected. The formula also brings out the perhaps less obvious fact that congestion also increases if the *variability* of the service-time distribution is large, even if the mean service time stays the same. Intuitively, this is because a highly variable service-time random variable will have a greater chance of taking on a large value (since it must be positive), which means that the (single) server will be tied up for a long time, causing the queue to build up.